

同济大学计算机系
计算机组成原理课程设计实验报告



题目：54 条 MIPS 指令 CPU 设计与应用验证

学 号 2252431
姓 名 孙骁远
专 业 计算机科学与技术
授课老师 秦国锋

日期：2024.07.13

一、 实验环境部署与硬件配置说明

使用 vivado 2016.2 作为开发软件搭配 NEXYS 4 DDR Atrix-7 开发板

二、 实验的总体结构

1、54 条 MIPS 指令 CPU 的总体结构

CPU 主要由数据通路和控制部件两部分组成。数据通路是指是指在指令执行过程中，数据所经过的路径和路径上所涉及的功能部件，涉及但不限于：

- 寄存器组：用于存储指令执行过程中的临时数据和变量。
- 算术逻辑单元：用于执行算术和逻辑运算。
- 内存接口：用于 CPU 与内存之间的数据交换。

而控制部件根据每条指令的不同功能，控制取指令、分析指令、执行指令等操作保证正确的指令执行流程。

CPU 功能主要包括以下几点

- (1) 指令控制：完成取指令、分析指令和执行指令的操作。这是 CPU 最基本的功能，确保程序能够按照预定的顺序执行。
- (2) 操作控制：它管理每条指令的操作信号，把信号送到相应的部件，从而控制这些部件按指令完成动作。
- (3) 时间控制：为每条指令按时间顺序提供应有的控制信号。这确保了指令执行的同步性和时序性。
- (4) 数据加工：对数据进行算数、逻辑计算。这是 CPU 的核心功能之一，涉及到 ALU 的操作。
- (5) 中断处理：对特殊请求进行处理，防止异常情况造成的损失。中断机制允许 CPU 响应外部或内部的紧急事件。这也是 54 条指令 CPU 不同于 31 条指令 CPU 的操作。

本次实验是 54 指令单周期 CPU，所有指令都在一个时钟周期内完成。在每条指令执行过程中数据通路的任何资源都不能并行利用，因此，任何需要多次使用的资源都需要设置多个。执行过程大致可分为取指周期、指令译码、间址周期、执行周期四个阶段。

2、指令格式

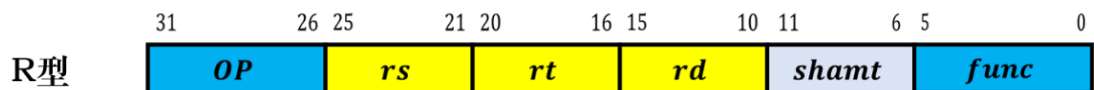
指令是一组有意义的二进制代码，负责控制机器语言，分为操作码和地址码两部分。共有 54 条，每条的指令说明、指令格式、16 进制指令码都在下图中给出

指令	指令说明	指令格式	OP 31-26	FUNCT 5-0	指令码 16 进制
addi	加立即数	addi rt, rs, immediate	001000		20000000
addiu	加立即数（无符号）	addiu rd, rs, immediate	001001		24000000
andi	立即数与	andi rt, rs, immediate	001100		30000000
ori	或立即数	ori rt, rs, immediate	001101		34000000
sltiu	小于立即数置 1（无符号）	sltiu rt, rs, immediate	001011		2C000000
lui	立即数加载高位	lui rt, immediate	001111		3C000000
xori	异或（立即数）	xori rt, rs, immediate	001110		38000000
slti	小于置 1（立即数）	slti rt, rs, immediate	001010		28000000
addu	加（无符号）	addu rd, rs, rt	000000	100001	00000021
and	与	and rd, rs, rt	000000	100100	00000024
beq	相等时分支	beq rs, rt, offset	000100		10000000
bne	不等时分支	bne rs, rt, offset	000101		14000000
j	跳转	j target	000010		08000000
jal	跳转并链接	jal target	000011		0C000000
jr	跳转至寄存器所指地址	jr rs	000000	001000	00000009
lw	取字	lw rt, offset(base)	100011		8C000000
xor	异或	xor rd, rs, rt	000000	100110	00000026
nor	或非	nor rd, rs, rt	000000	100111	00000027
or	或	or rd, rs, rt	000000	100101	00000025
sll	逻辑左移	sll rd, rt, sa	000000	000000	00000000

sllv	逻辑左移（位数可变）	sllv rd, rt, rs	000000	000100	00000004
sltu	小于置 1（无符号）	sltu rd, rs, rt	000000	101011	0000002B
sra	算数右移	sra rd, rt, sa	000000	000011	00000003
srl	逻辑右移	srl rd, rt, sa	000000	000010	00000002
subu	减（无符号）	sub rd, rs, rt	000000	100010	00000022
sw	存字	sw rt, offset(base)	101011		AC000000
add	加	add rd, rs, rt	000000	100000	00000020
sub	减	sub rd, rs, rt	000000	100010	00000022
slt	小于置 1	slt rd, rs, rt	000000	101010	0000002A
srlv	逻辑右移（位数可变）	srlv rd, rt, rs	000000	000110	00000006
srav	算数右移（位数可变）	srav rd, rt, rs	000000	000111	00000007
clz	前导零计数	clz rd, rs	011100	100000	70000020
divu	除（无符号）	divu rs, rt	000000	011011	0000001B
eret	异常返回	eret	010000	011000	42000018
jalr	跳转至寄存器所指地址，返回地址保存在	jalr rs	000000	001001	00000008

lb	取字节	lb rt, offset(base)	100000		80000000
lbu	取字节（无符号）	lbu rt, offset(base)	100100		90000000
lhu	取半字（无符号）	lhu rt, offset(base)	100101		94000000
sb	存字节	sb rt, offset(base)	101000		A0000000
sh	存半字	sh rt, offset(base)	101001		A4000000
lh	取半字	lh rt, offset(base)	100001		84000000
mfc0	读 CP0 寄存器	mfc0 rt, rd	010000	000000	40000000
mfhi	读 Hi 寄存器	mfhi rd	000000	010000	00000010
mflo	读 Lo 寄存器	mflo rd	000000	010010	00000012
mtc0	写 CP0 寄存器	mtc0 rt, rd	010000	000000	40800000
mthi	写 Hi 寄存器	mthi rd	000000	010001	00000011
mtlo	写 Lo 寄存器	mtlo rd	000000	010011	00000013
mul	乘	mul rd, rs, rt	011100	000010	70000002
multu	乘（无符号）	multu rs, rt	000000	011001	00000019
syscall	系统调用	syscall	000000	001100	0000000C
teq	相等异常	teq rs, rt	000000	110100	00000034
bgez	大于等于 0 时分支	bgez rs, offset	000001		04010000
break	断点	break	000000	001101	0000000D
div	除	div rs, rt	000000	011010	0000001A

与 31 条一样，也可以分为三种指令类型：R 型、I 型、J 型



其中：

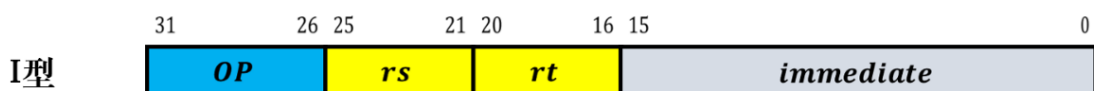
op: 31 位~26 位，6 位操作码，决定执行何种指令，R 型指令的操作码均为 000000，其余指令操作码各不相同。

rs, rt: 5 位地址码，两个源操作数所在的寄存器号

rd: 5 位地址码，目标操作数所在的寄存器号

shamt: 5 位数据，专用于移位指令中代表位移量。

func: 6 位功能码，与操作码配合使用，在 R 类指令中，操作码均为 0，用不同的功能码区分执行何种指令。



其中：

op: 31 位~26 位，6 位操作码，决定执行何种指令，R 型指令的操作码均为 000000，其余指令操作码各不相同。

rs, rt: 5 位地址码，两个源操作数所在的寄存器号

rd: 5 位地址码，目标操作数所在的寄存器号

immediate: 用作无符号的逻辑操作数、有符号的算数操作数、数据加载，数据保存指令的

数据地址字节偏移量和分支指令中相对程序计数器的有符号偏移量。



其中：

OP：操作码，决定操作功能

address：转移地址（立即数）

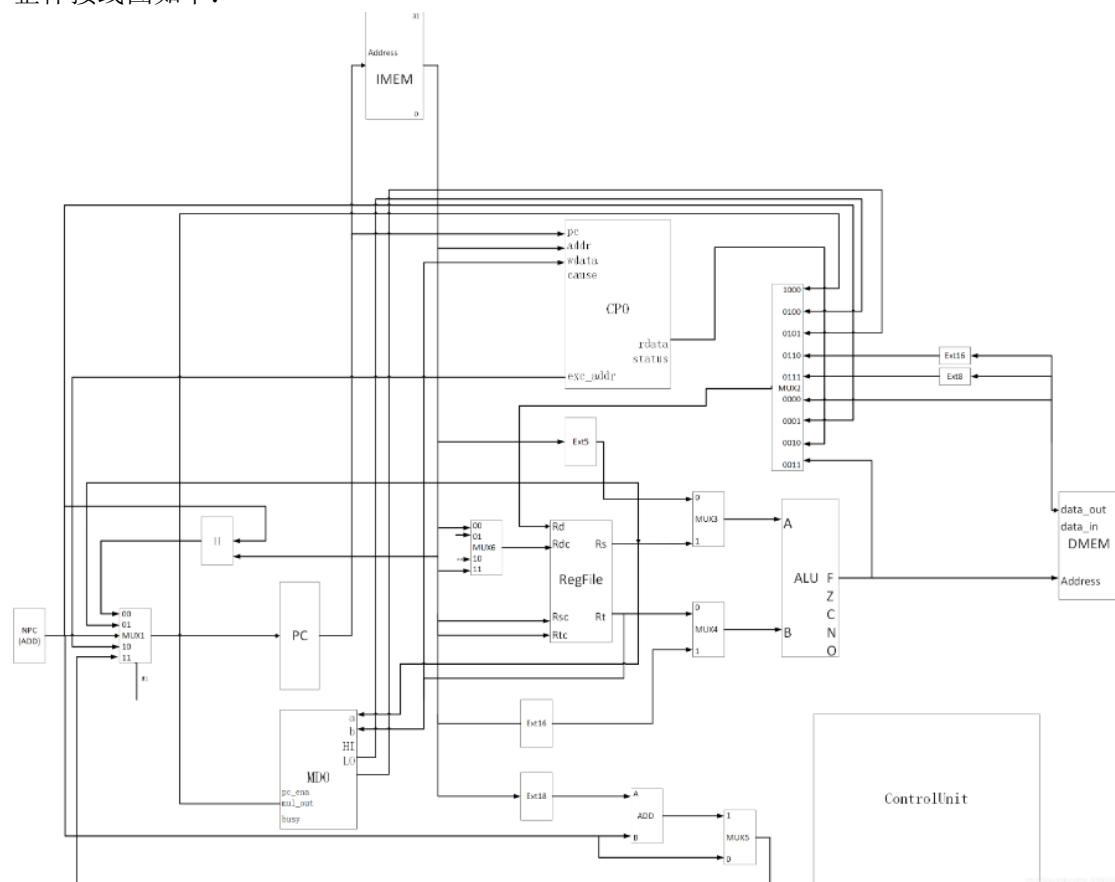
三、 总体架构部件的解释说明

1、CPU 总体结构部件的解释说明

主要结构部件有以下几种

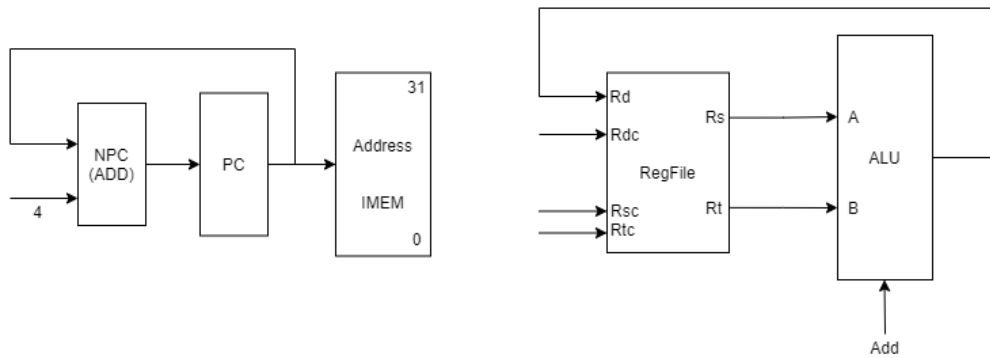
1. 程序计数器（PC）：存储下一条要执行指令的地址
2. 指令寄存器（IR）：暂存当前正在执行的指令
3. 指令内存（IMEM）：存储程序的指令代码
4. 数据内存（DMEM）：存储数据，如变量和数组
5. 算术逻辑单元（ALU）：执行算术运算和逻辑运算
6. 寄存器堆（regfile）：包含一组通用寄存器，用于存储指令执行过程中的变量和中间结果
7. 多路选择器（MUX）：用于在不同的信号源之间选择数据
8. 中断控制器（CP0）：管理中中断请求和处理，允许 CPU 响应紧急事件

整体接线图如下：

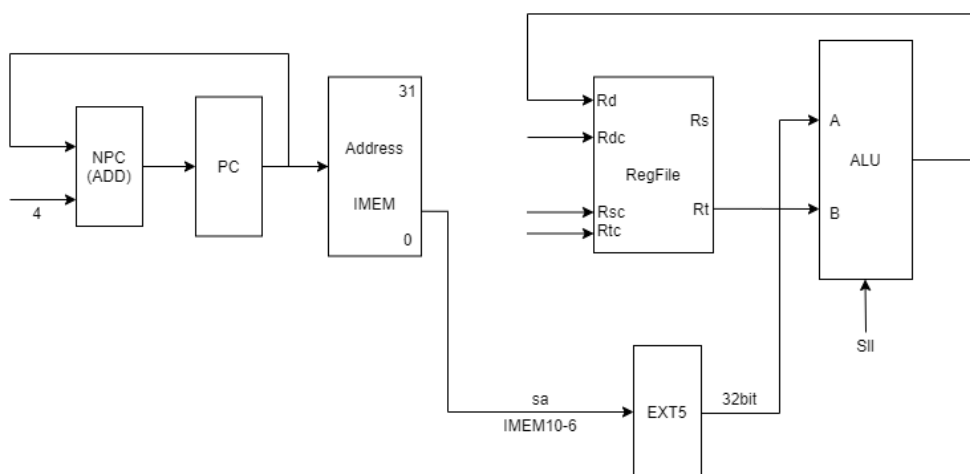


2、数据通路图设计（对于相似的指令，相同的数据通路图不再重复画出）：

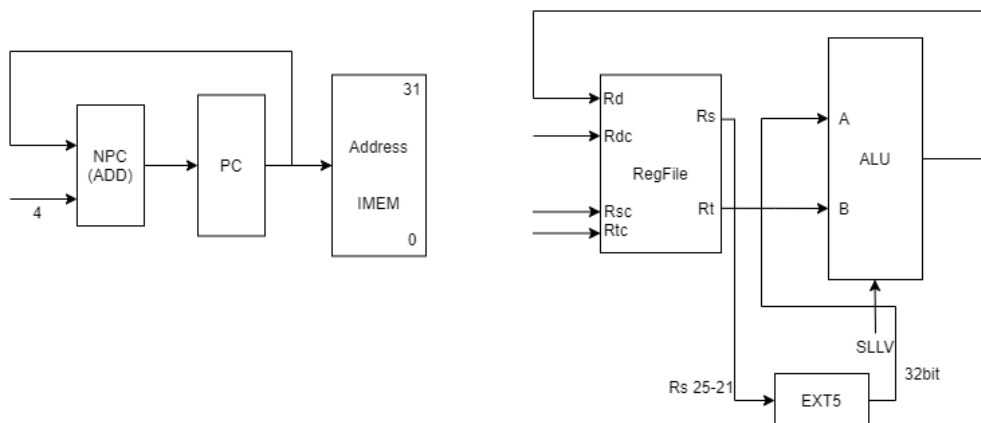
（1） ADD/ADDU/SUB/SUBU/AND/OR/XOR/NOR/SLT/SLTU



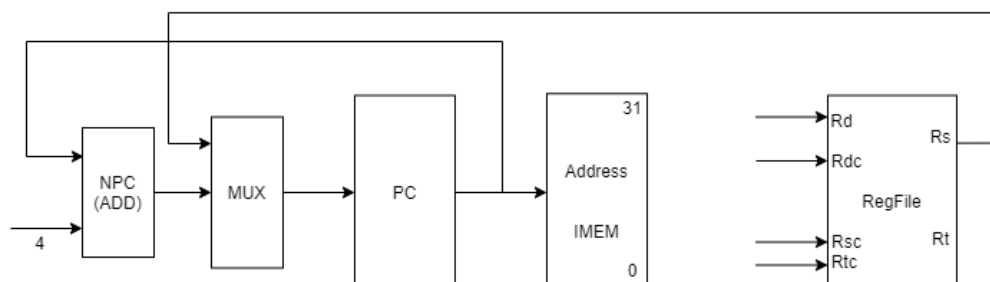
（2） SLL/SRL/SRA



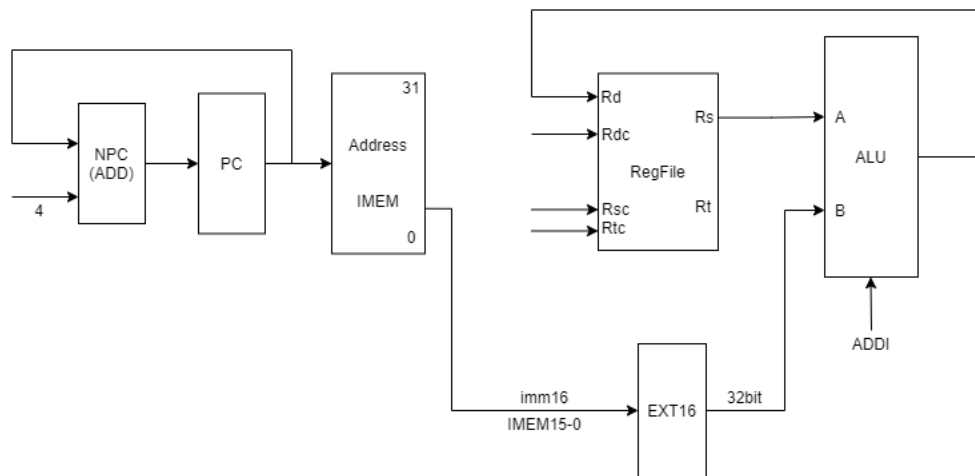
（3） SLLV/SRLV/SRAV



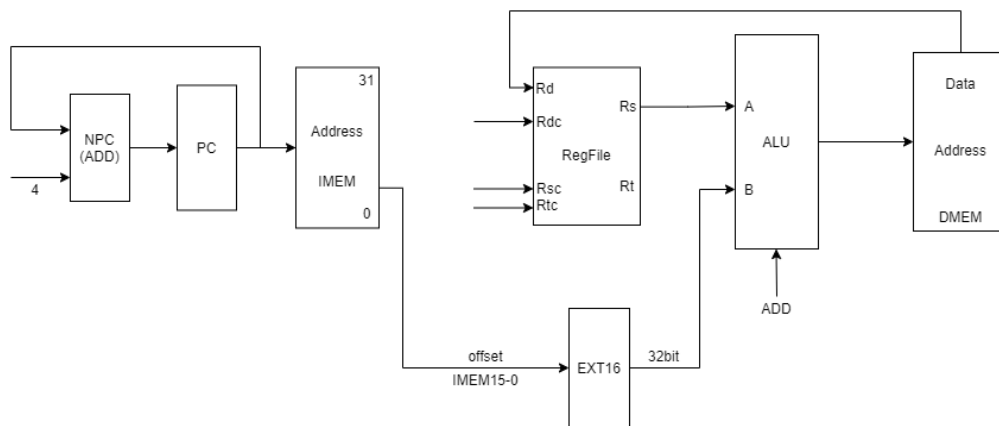
（4） JR



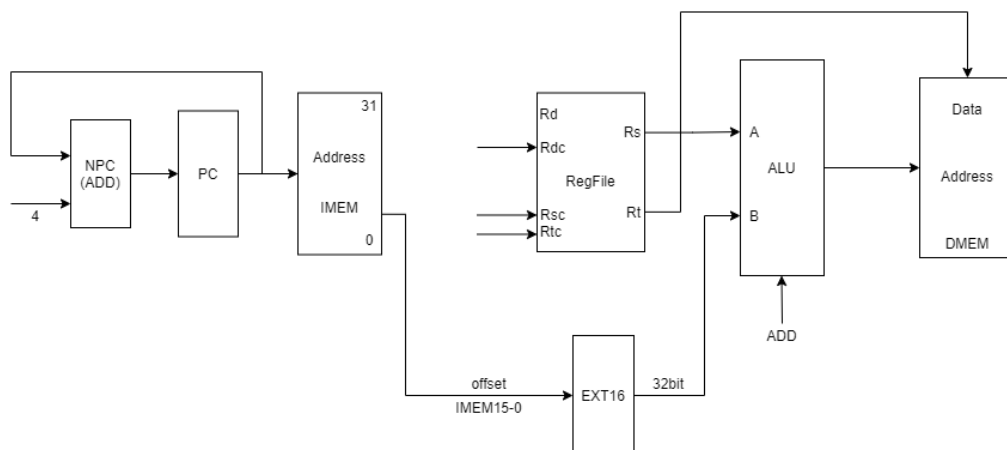
(5) ADDI/ADDIU/ANDI/ORI/XORI



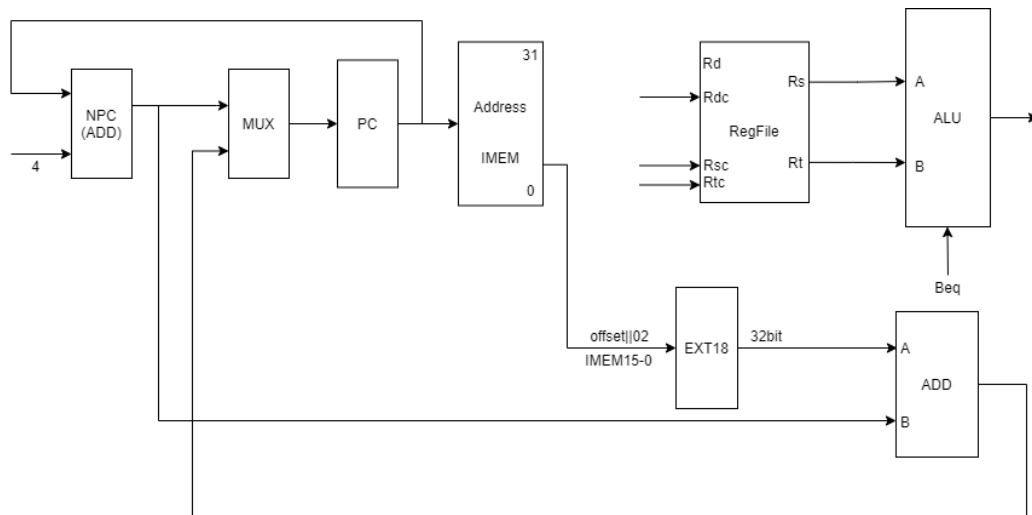
(6) LW



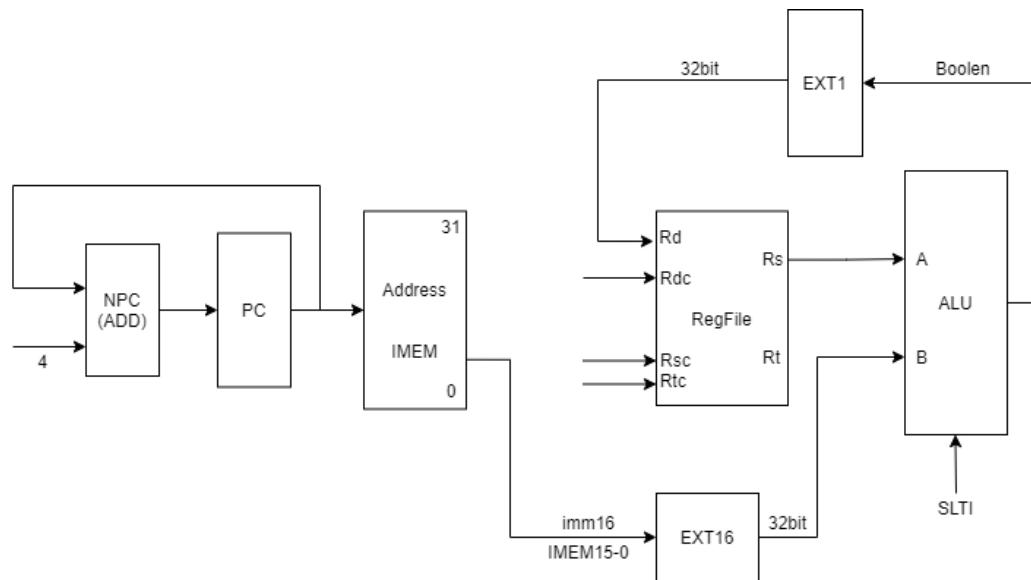
(7) SW



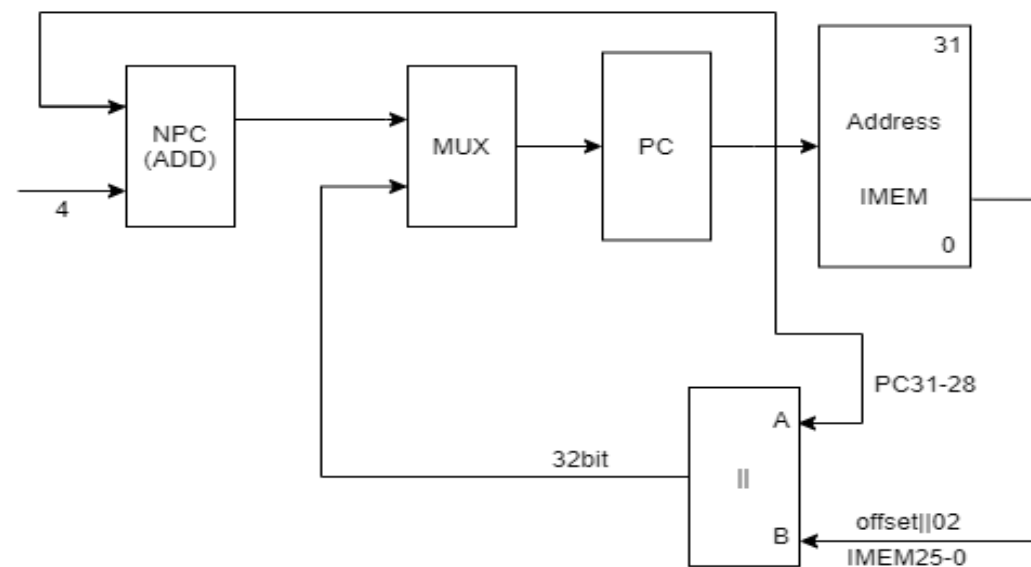
(8) BEQ/BNE



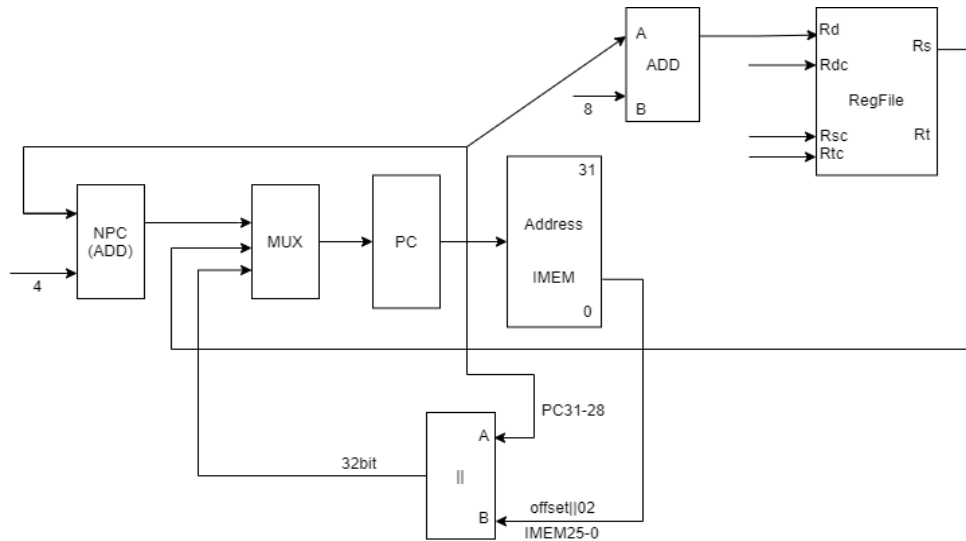
(9) SLTI/SLTIU/LUI



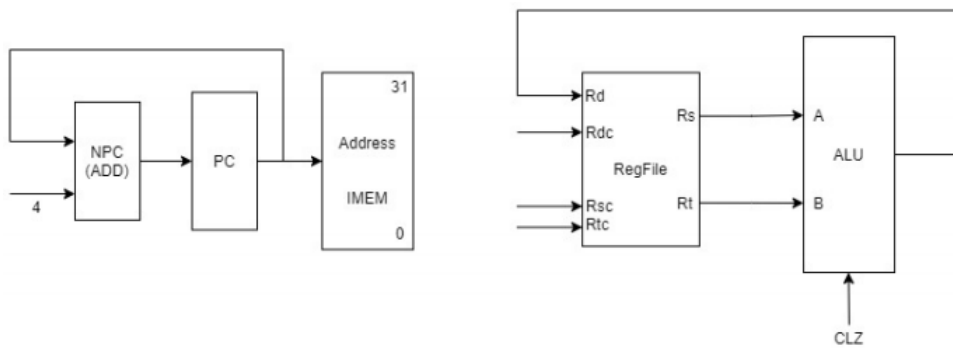
(10) J



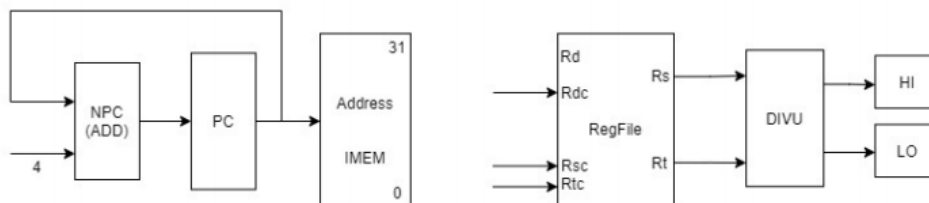
(11) JAL



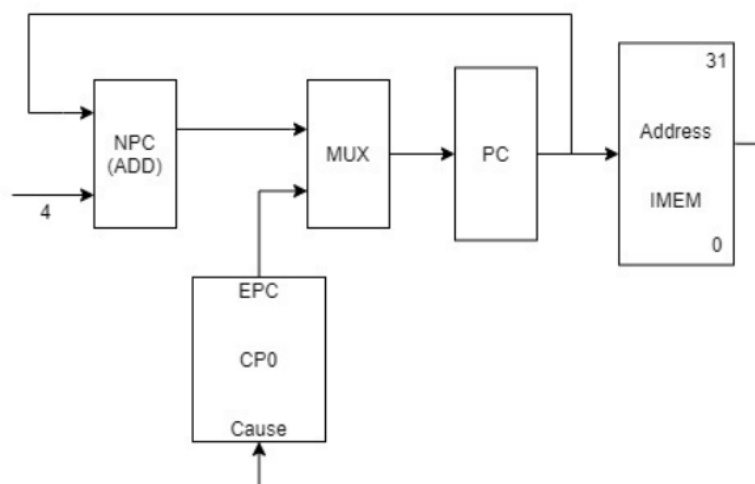
(12) CLZ



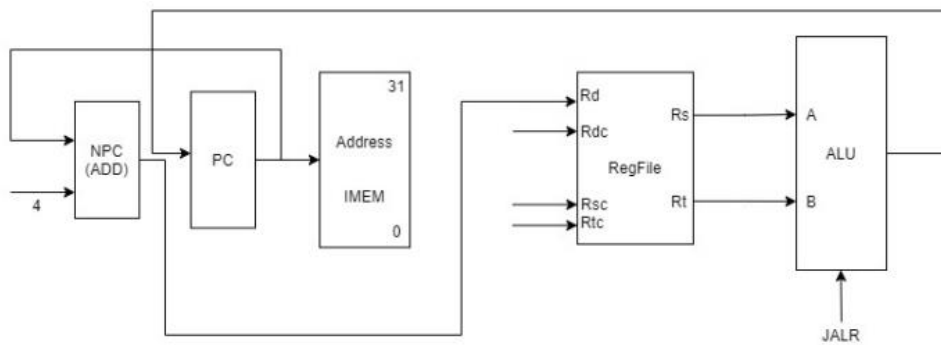
(13) DIVU



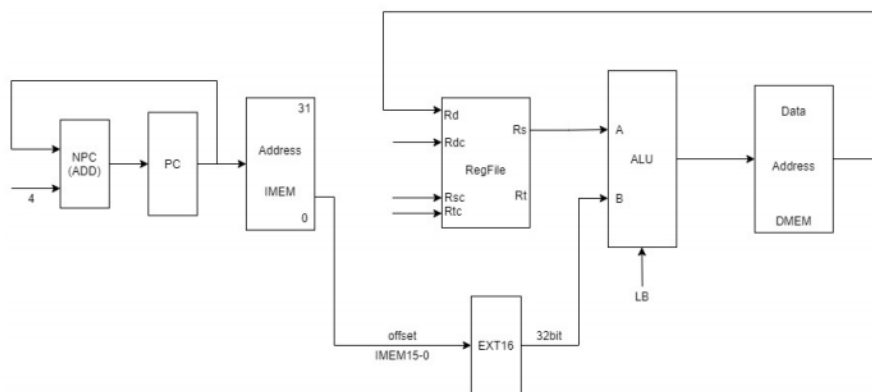
(14) ERET



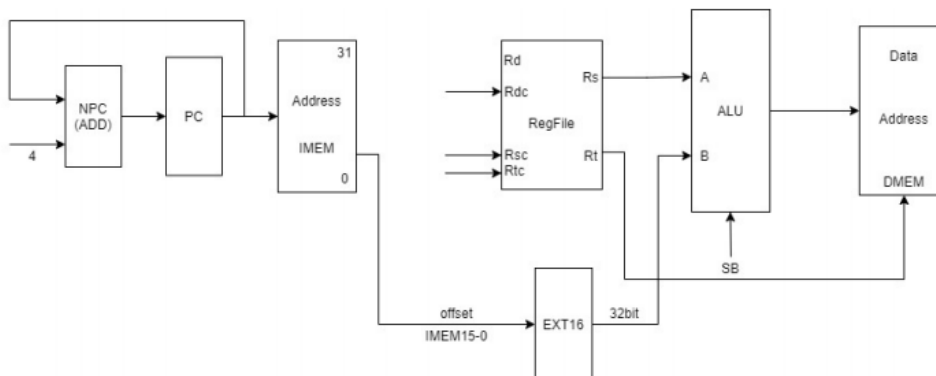
(15) JALR



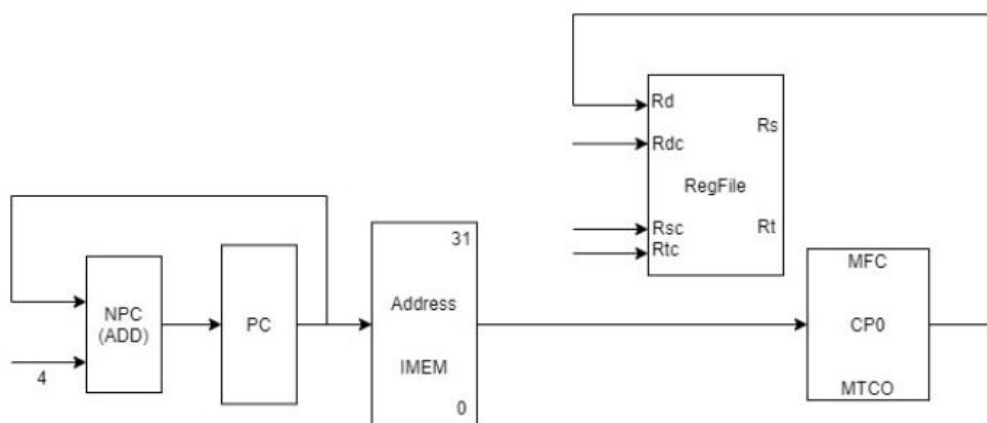
(16) LB/LBU/LHU/LH (这里向 ALU 输入的指令分别是 LB, LBU, LHU, LH, 图中只用 LB 表示)



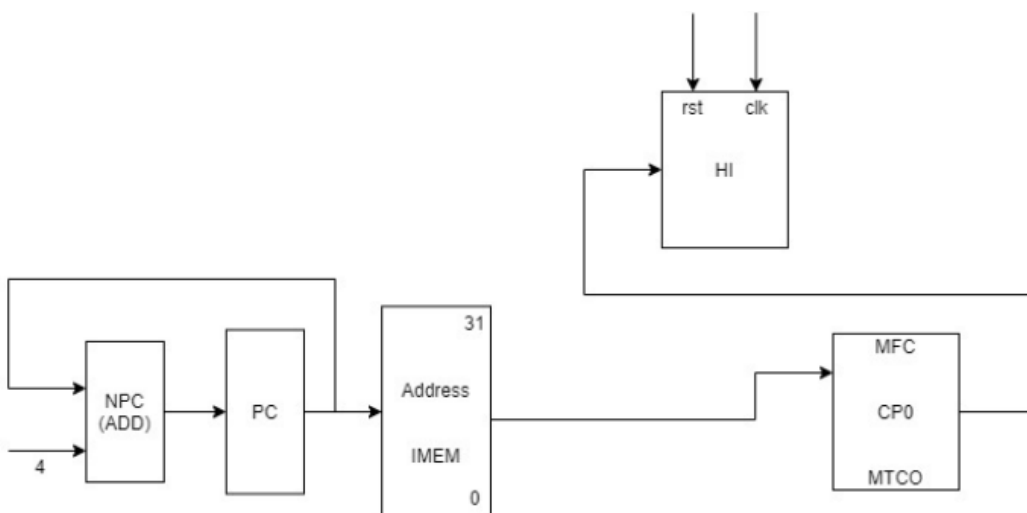
(17) SB/SH (这里向 ALU 输入的指令分别是 SB, SH, 其余接线完全相同, 图中只用 SB 表示)



(18) MFC0/MTC0



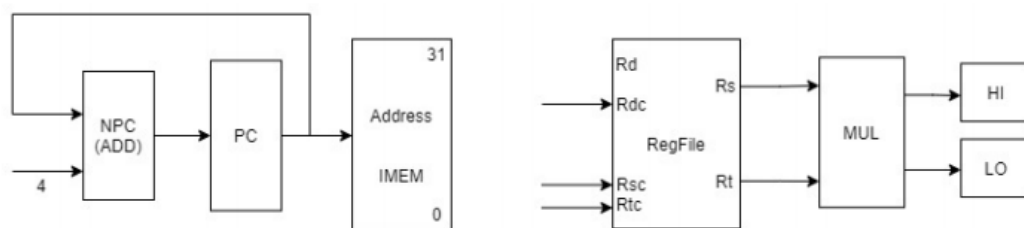
(19) MFHI/MFLO (这里上面的寄存器应该分别标 HI 和 LO, 其余接线一致, 用 MFHI 的接线表示)



(20) MTHI/MTLO



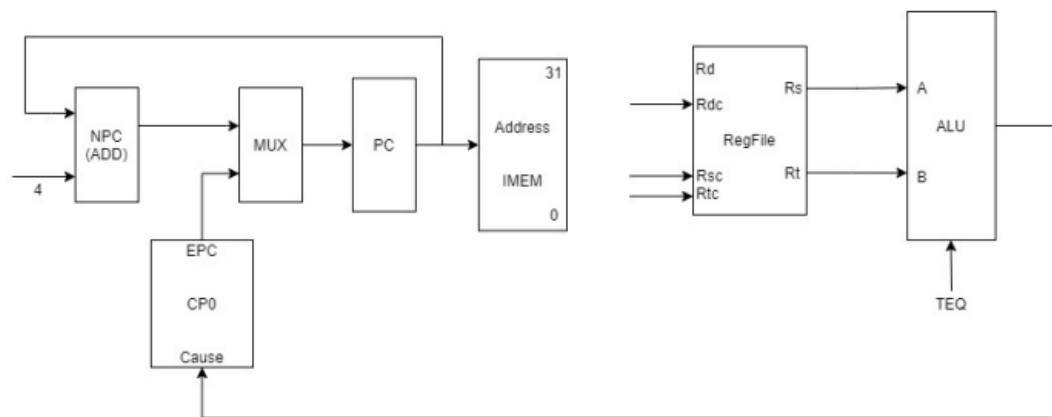
(21) MUL



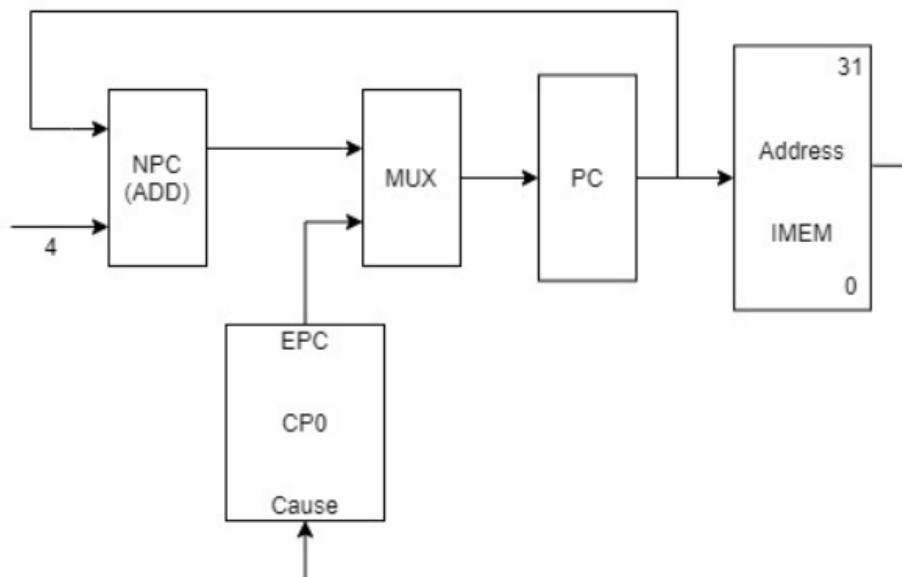
The diagram illustrates the RISC-V processor core architecture. It consists of the following components and connections:

- NPC (ADD)**: Next-PC (Addend) block. It receives a 4-bit input from the left and a feedback signal from the IMEM block. Its output goes to the MUX.
- MUX**: Multiplexer block. It receives inputs from the NPC (ADD) block and the EPC block. Its output goes to the PC.
- PC**: Program Counter block. It receives input from the MUX and outputs to the IMEM block. A feedback signal from the IMEM block goes back to the NPC (ADD) block.
- IMEM**: Instruction Memory block. It receives a 32-bit address from the PC and outputs a 32-bit instruction. A feedback signal from the IMEM block goes back to the NPC (ADD) block.
- EPC**: Exception Program Counter block. It receives input from the MUX and outputs to the MUX. It also receives input from the Cause register.
- CP0**: Control and Status Register (CSR) block. It receives input from the Cause register and outputs to the Cause register.
- Cause**: Exception Cause register. It receives input from the CP0 block and outputs to the EPC block.

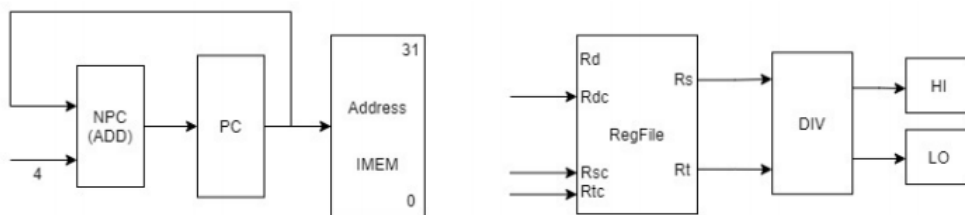
(25) TEQ



(26) BREAK

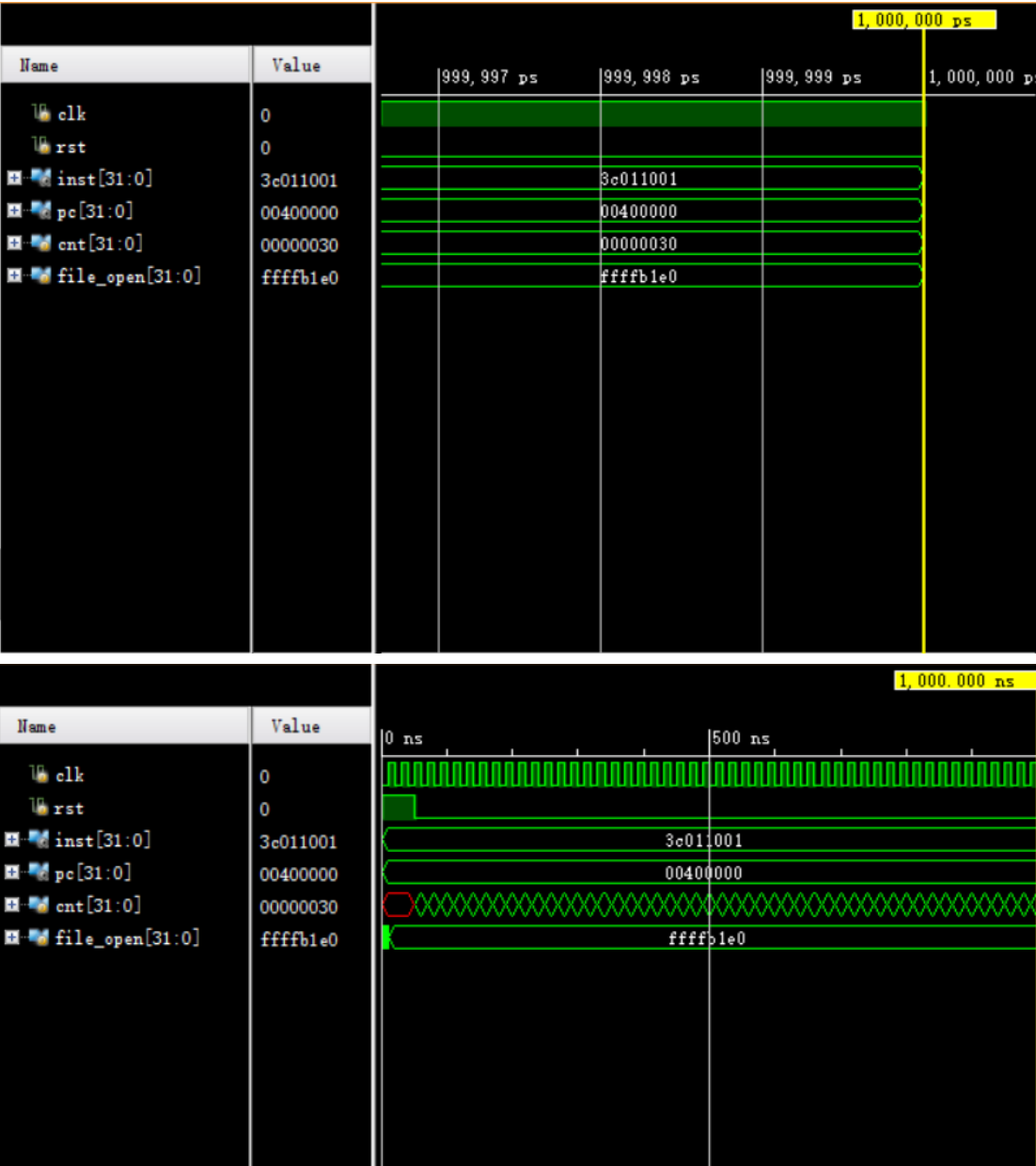


(27) DIV



四、 实验仿真过程

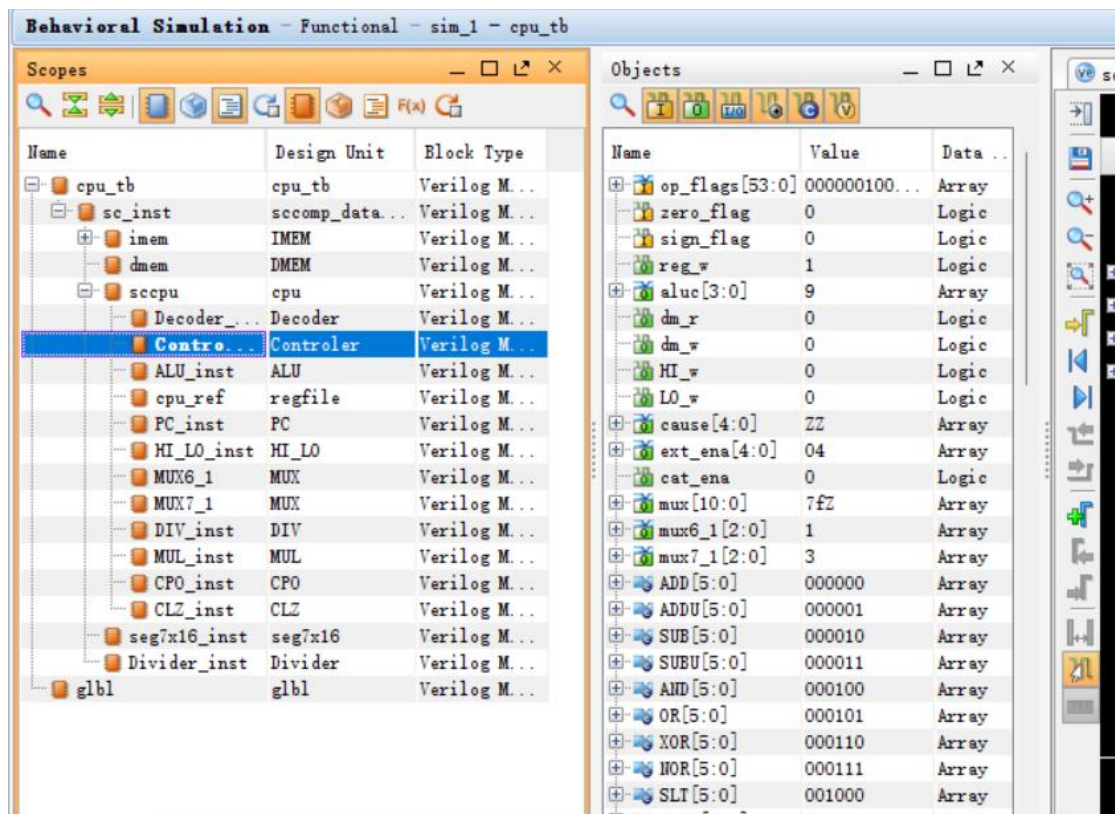
1、 Run simulation 仿真过程，vivado 仿真界面



五、

1、CPU 的仿真过程

可以通过仿真界面中选择不同子模块查看任意一个寄存器的值，如下图所示



六、 实验仿真的波形图及某时刻寄存器值的物理意义

仿真波形图及某时刻寄存器值的物理意义

根据以上仿真界面，其中变量的含义如下：

clk:时钟信号，频率为 50MHz

rst:复位信号，初始化系统，确保从已知状态开始仿真

inst: 要执行的指令码，通过\$fdisplay 函数将指令地址和寄存器值写入文件

pc:下一条指令地址

cnt:计数器，记录已经执行几条指令

file_open:打开文件的句柄

仿真文件将通用寄存器的值写入文件，每个寄存器的值表示某一时刻的数据状态，例如\$zero: 始终为 0 的寄存器

\$at: 保留给汇编器临时使用

\$v0,\$v1: 用于存储函数返回值

\$a0-\$a3: 用于传递函数参数

\$t0-\$t9: 临时寄存器，用于存储中间结果

这些寄存器的值在仿真过程中会随着指令的执行而变化，反映了程序的执行状态。

七、 实验验算数学模型及算法程序

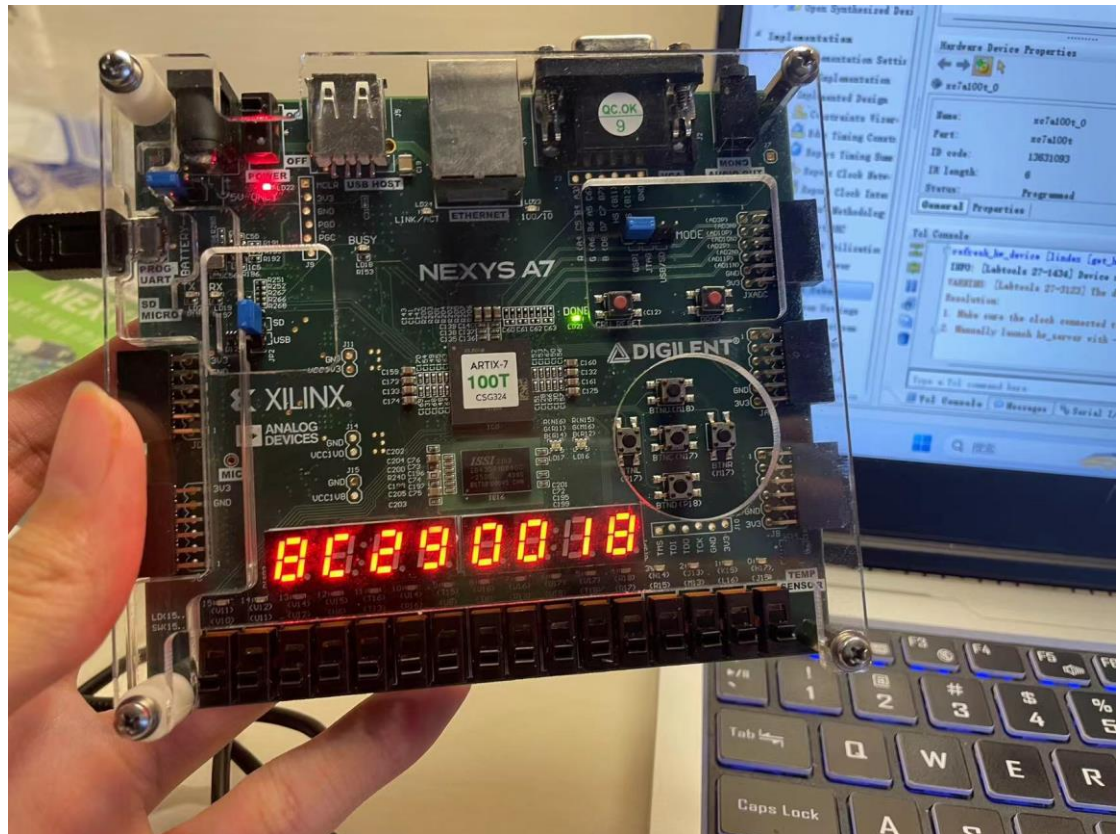
比萨塔摔鸡蛋游戏模型。两个同学在可变换层数的比萨塔上摔鸡蛋，一个同学秘密设定同一批鸡蛋耐摔值；另一个同学在指定层高的比萨塔拿着鸡蛋往下摔，用最少的摔次数和摔破的鸡蛋数求出鸡蛋的耐摔值。假定在耐摔值的楼层及其下面楼层，鸡蛋摔不破，可以重复使用，否则鸡蛋摔破。要求模型的算法输出包括：摔的总次数、摔的总鸡蛋数、最后摔的鸡蛋是否摔破。请使用 MIPS 指令汇编设计该验证模型的算法，并利用编译器生成 MIPS 指令

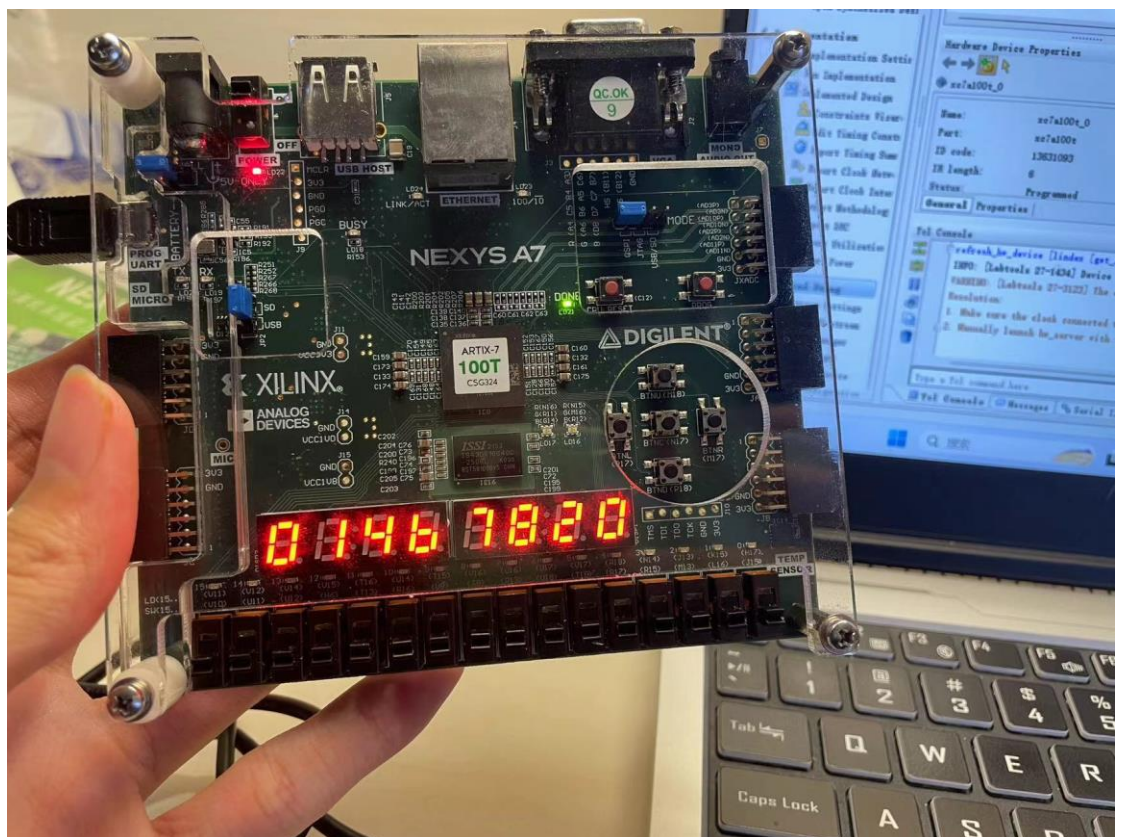
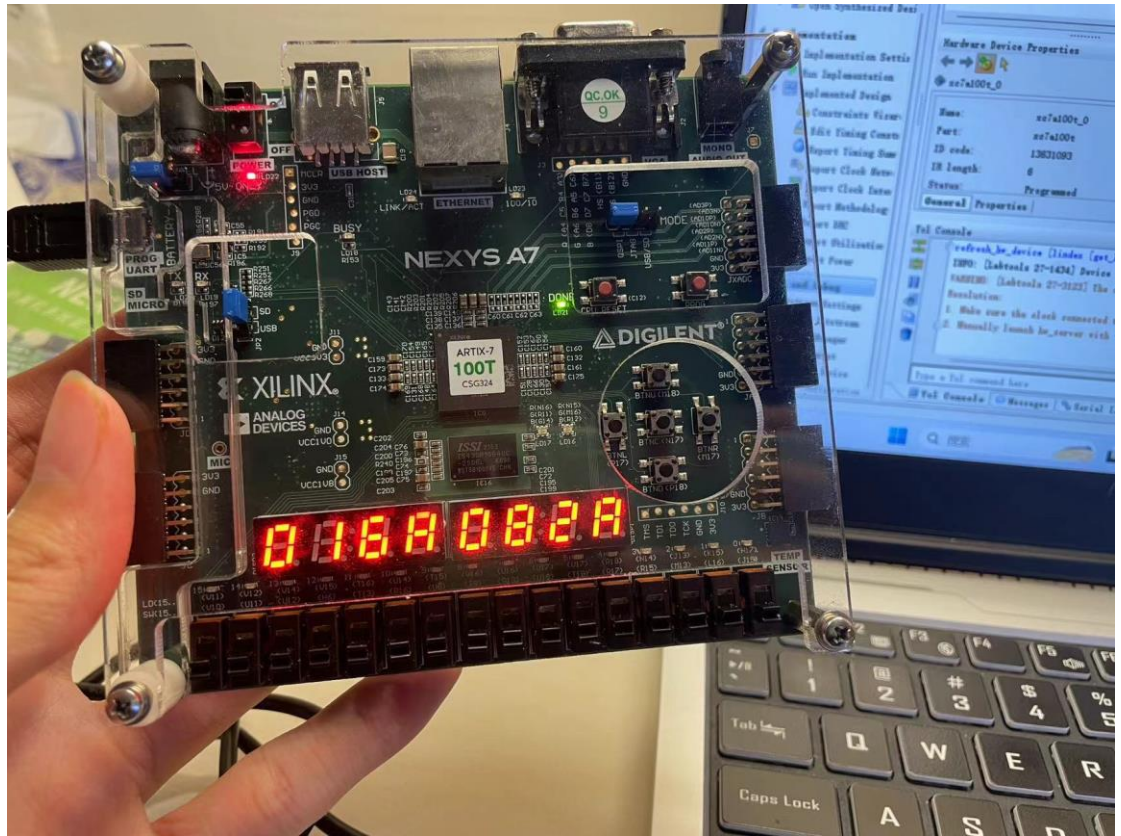
集可执行目标程序。

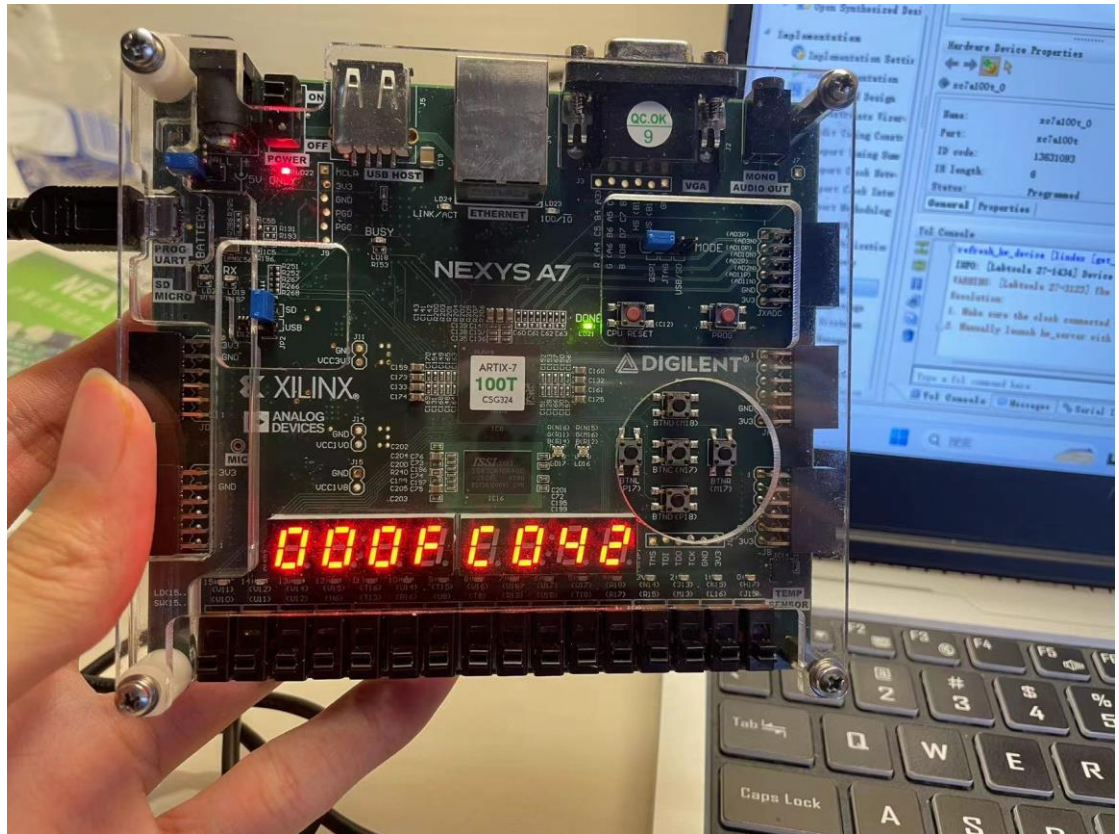
该程序运行时，按键模拟暂停运行和恢复运行，以及按键模拟停止运行等中断。

八、实验验算程序下板测试过程与实现

下板图如下，七段数码管不断切换指令，说明 CPU 正在有序读取、处理指令，它的功能基本正常。







输出结果显示如下：

```
int attempts = 0; // 尝试次数
int brokenEggs = 0; // 摔碎的鸡蛋数
int isBroken = 0; // 最后摔的鸡蛋是否碎掉(0表示没摔碎, 1表示摔碎)

printf("Enter the egg strength: ");
scanf("%d", &eggStrength); // 输入鸡蛋的耐摔值

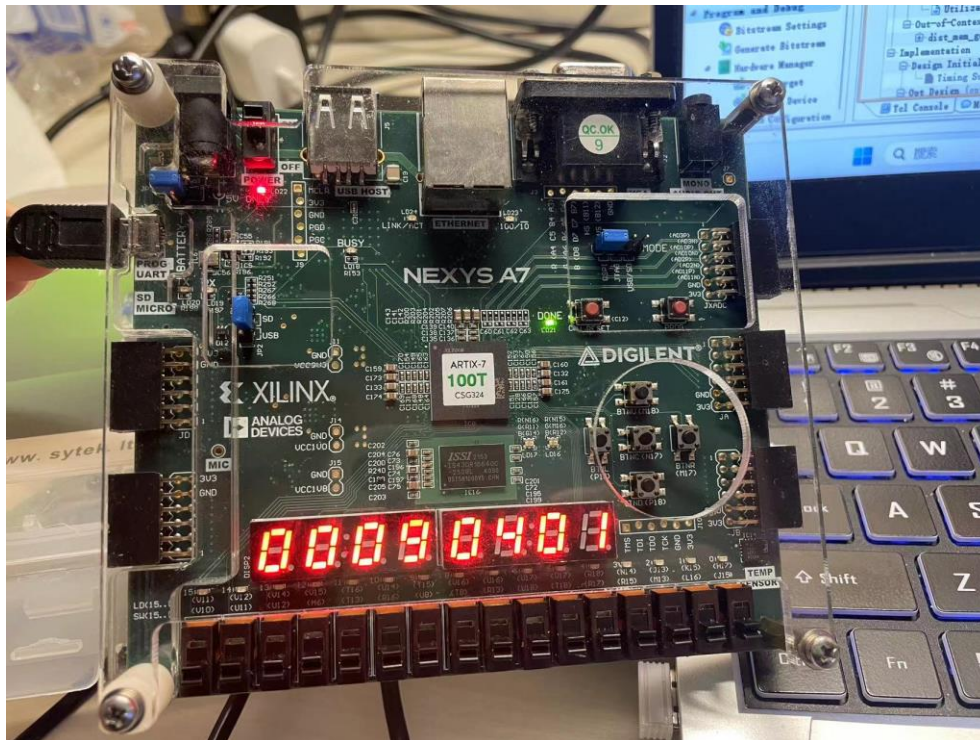
while (lowerBound <= upperBound) {
    int currentFloor = (lowerBound + upperBound) / 2; // 当前尝试的楼层
}

Microsoft Visual Studio 调试 × + v

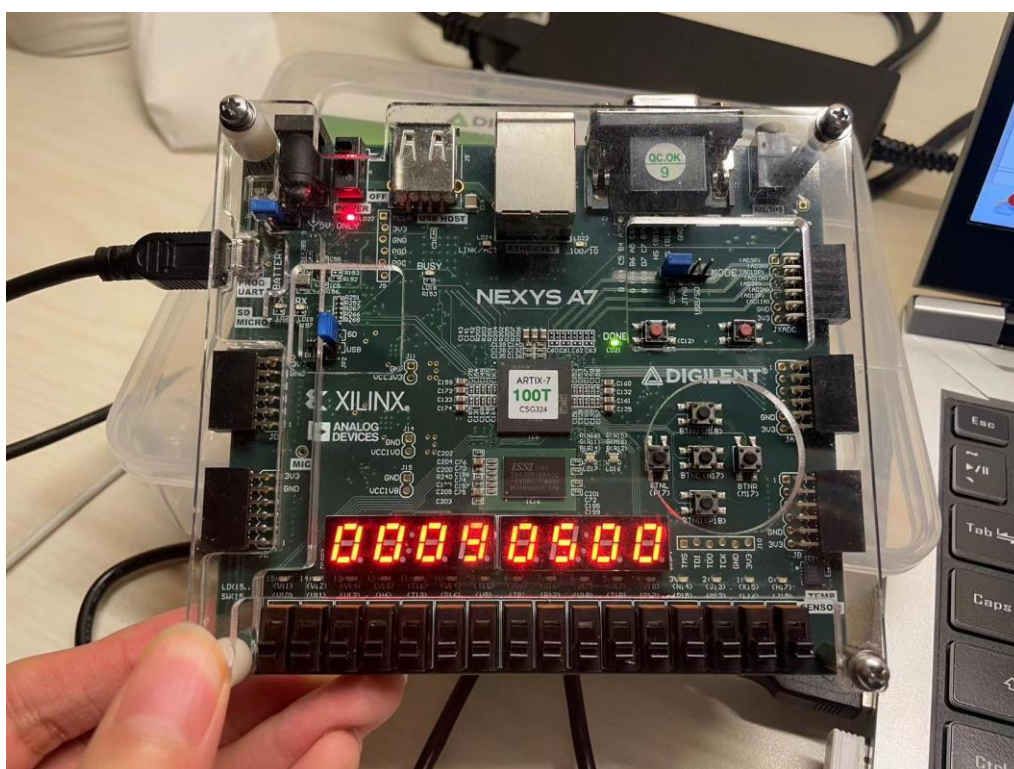
Enter the egg strength: 400
Minimum attempts: 9
Broken eggs: 4
Is the last egg broken? 1

D:\摔鸡蛋程序\x64\Debug\摔鸡蛋程序.exe (进程 17844)已退出, 代码为 0。
按任意键关闭此窗口...
```

我设定数码管前两位恒为0, 第34, 第56, 第78位分别表示最小尝试次数, 摔碎的鸡蛋数, 最后一次是否摔碎(1表示摔碎, 0表示没摔碎)



```
Microsoft Visual Studio 调试 × +  
Enter the egg strength: 50  
Minimum attempts: 9  
Broken eggs: 5  
Is the last egg broken? 0  
  
D:\摔鸡蛋程序\x64\Debug\摔鸡蛋程序.exe (进程 24852)已退出，代码为 0。  
按任意键关闭此窗口。 . . |
```



九、 总结与体会

54 条 CPU 是在 31 条 CPU 的基础上增加了中断处理，乘法器，除法器，前导零计数器等功能。实验过程中，我深入学习了 CPU 的工作原理，包括指令集、数据通路、控制信号以及异常处理等关键概念。从无到有搭建了 CPU 的各个组成部分，包括包括指令集、寄存器、算术逻辑单元（ALU）、控制单元以及内存接口等，并且能模拟执行一系列指令。

首先通过每条指令所需的部件画出数据通路，发现很多指令的数据通路是完全一样的；然后画出指令流程图进行 CPU 控制器的设计。前期准备工作完成后，开始代码环节。通过顶层模块调用 CPU，指令寄存器（设置 IP 核）和数据寄存器。在 CPU 中需要 ALU，regfile，PC 等模块实现 CPU 的功能。代码完成后，通过 IP 核读入 coe 文件，将寄存器结果写入文件与 MARS 编译后的结果对比，准确无误说明设计正确。在设计过程中遇到很多错误，主要是比对过程中寄存器结果、地址、程序计数器与 MARS 汇编的结果不一样。检查发现数据通路，微指令的逻辑表达式有误，重新分析、设计后解决了这些问题。

总之，CPU 设计是一个不断发展的领域。我通过模块化的设计方式和不断前后仿真实现了稍微复杂的 CPU 设计，激发了我继续探索和学习硬件的热情。

十、 附件（所有程序）

CPU 的设计程序

sccomp_dataflow 模块，顶层模块，调用 cpu，IMEM，DMEM，seg7x16，Divider 五个模块

```
module sccomp_dataflow(
    input clk_in,          //时钟信号
    input reset,           //复位信号
    //output [31:0] inst,
    //output [31:0] pc
    //下板用
    output [7:0] o_seg, //输出内容
    output [7:0] o_sel //片选信号
);

wire [31:0] pc_out;
wire [31:0] dm_addr_temp;
wire [31:0] im_addr_in;
wire [31:0] im_instr_out;
wire dm_ena;
wire dm_r, dm_w;
wire [31:0] dm_addr;
wire [31:0] dm_data_out;
wire [31:0] dm_data_w;

wire sb_flag;
```

```

wire sh_flag;
wire sw_flag;
wire lb_flag;
wire lh_flag;
wire lbu_flag;
wire lhu_flag;
wire lw_flag;

assign im_addr_in = pc_out - 32'h00400000;
assign dm_addr = (dm_addr_temp - 32'h10010000)/4;
assign pc = pc_out;
assign inst = im_instr_out;

```

/*实例化*/

```

IMEM imem(
    .im_addr_in(im_addr_in[12:2]),
    .im_instr_out(im_instr_out)
);

```

```

DMEM dmem(
    .dm_clk(clk_cpu),
    .dm_ena(dm_ena),
    .dm_r(dm_r),
    .dm_w(dm_w),
    .sb_flag(sb_flag),
    .sh_flag(sh_flag),
    .sw_flag(sw_flag),
    .lb_flag(lb_flag),
    .lh_flag(lh_flag),
    .lbu_flag(lbu_flag),
    .lhu_flag(lhu_flag),
    .lw_flag(lw_flag),
    .dm_addr(dm_addr[6:0]),
    .dm_data_in(dm_data_w),
    .dm_data_out(dm_data_out)
);

```

/*实例化 CPU*/

```

cpu sccpu(
    .clk(clk_cpu),
    .ena(1'b1),
    .rst(reset),
    .instr_in(im_instr_out),

```

```

        .dm_data(dm_data_out),
        .dm_ena(dm_ena),
        .dm_w(dm_w),
        .dm_r(dm_r),
        .pc_out(pc_out),
        .dm_addr(dm_addr_temp),
        .dm_data_w(dm_data_w),
        .sb_flag(sb_flag),
        .sh_flag(sh_flag),
        .sw_flag(sw_flag),
        .lb_flag(lb_flag),
        .lh_flag(lh_flag),
        .lbu_flag(lbu_flag),
        .lhu_flag(lhu_flag),
        .lw_flag(lw_flag)
    );
/*实例化 7 段数码管*/
seg7x16 seg7x16_inst(
    .clk(clk_in),
    .reset(reset),
    .cs(1'b1),
    .i_data(im_instr_out),
    .o_seg(o_seg),
    .o_sel(o_sel)
);

/*实例化分频器*/
Divider Divider_inst(
    .clk(clk_in),
    .rst_n(~reset),
    .clk_out(clk_cpu)
);

Endmodule

```

IMEM, 指令存储器, 通过输入一个 11 位地址, 从内部存储单元中读取相应 32 位指令码

```

module IMEM(
    input [10:0] im_addr_in,
    output [31:0] im_instr_out
);

```

```

/* 实例化 IP 核 */
dist_mem_gen_0 imem(.a(im_addr_in),.spo(im_instr_out));
endmodule

```

DMEM, 数据存储单元, 异步读数据同步写数据

```

module DMEM(
    input dm_clk,
    input dm_ena,
    input dm_r,
    input dm_w,
    input sb_flag,
    input sh_flag,
    input sw_flag,
    input lb_flag,
    input lh_flag,
    input lbu_flag,
    input lhu_flag,
    input lw_flag,
    input [6:0] dm_addr,
    input [31:0] dm_data_in,
    output [31:0] dm_data_out
);

// DMEM 区域声明
reg [31:0] dmem[0:((1 << 7) - 1)]

// 根据读写信号和地址, 生成输出数据
wire [31:0] data_out;
assign dm_data_out = (dm_ena && dm_r && !dm_w) ? data_out : 32'b0;

// 写入数据到 DMEM
always @(posedge dm_clk) begin
    if (dm_ena && dm_w && !dm_r) begin
        case ({sw_flag, sh_flag, sb_flag})
            3'b001: dmem[dm_addr][7:0] <= dm_data_in[7:0]; // SB
            3'b010: dmem[dm_addr >> 1][15:0] <= dm_data_in[15:0]; // SH
            3'b100: dmem[dm_addr >> 2] <= dm_data_in; // SW
            default: ; // 其他情况不写入
        endcase
    end
end
end

```

```

// 根据读取信号和地址，从 DMEM 读取数据
reg [31:0] data_out_reg;
always @(*) begin
    case ({lw_flag, lhu_flag, lh_flag, lb_flag, lbu_flag})
        5'b00001: data_out_reg = {24{dmem[dm_addr][7]}}; // LB
        5'b00010: data_out_reg = {24'b0, dmem[dm_addr][7:0]}; // LBU
        5'b00100: data_out_reg = {16{dmem[dm_addr >> 1][15]}}; // LH
        5'b01000: data_out_reg = {16'b0, dmem[dm_addr >> 1][15:0]}; // LHU
        5'b10000: data_out_reg = dmem[dm_addr >> 2]; // LW
        default: data_out_reg = 32'b0; // 其他情况返回 0
    endcase
end

// 将读取的数据输出
assign data_out = data_out_reg;
endmodule

```

CPU，实现 54 条 MIPS 指令的核心

```

module cpu(
    input clk,                //CPU 执行时钟
    input ena,                //使能信号端
    input rst,                //复位信号
    input [31:0] instr_in,    //当前要执行的指令
    input [31:0] dm_data,     //读取到的 DMEM 的具体内容
    output dm_ena,            //是否需要启用 DMEM
    output dm_w,              //如果启用 DMEM，是否为写入
    output dm_r,              //如果启用 DMEM，是否为读取
    output [31:0] pc_out,     //输出指令地址，告诉 IMEM 要取哪条
    output [31:0] dm_addr,    //启用 DMEM 的地址
    output [31:0] dm_data_w,  //要写入 DMEM 的内容
    output sb_flag,           //当前写入指令是否是 SB 发出
    output sh_flag,           //当前写入指令是否是 SH 发出
    output sw_flag,           //当前写入指令是否是 SW 发出
    output lb_flag,           //当前读取指令是否是 LB 发出
    output lh_flag,           //当前读取指令是否是 LH 发出
    output lbu_flag,          //当前读取指令是否是 LBU 发出
    output lhu_flag,          //当前读取指令是否是 LHU 发出
    output lw_flag,           //当前读取指令是否是 LW 发出
);

```


/* 54 条指令对应的序号 */

parameter ADD = 6'd0;
parameter ADDU = 6'd1;
parameter SUB = 6'd2;
parameter SUBU = 6'd3;
parameter AND = 6'd4;
parameter OR = 6'd5;
parameter XOR = 6'd6;
parameter NOR = 6'd7;
parameter SLT = 6'd8;
parameter SLTU = 6'd9;
parameter SLL = 6'd10;
parameter SRL = 6'd11;
parameter SRA = 6'd12;
parameter SLLV = 6'd13;
parameter SRLV = 6'd14;
parameter SRAV = 6'd15;
parameter JR = 6'd16;
parameter ADDI = 6'd17;
parameter ADDIU = 6'd18;
parameter ANDI = 6'd19;
parameter ORI = 6'd20;
parameter XORI = 6'd21;
parameter LW = 6'd22;
parameter SW = 6'd23;
parameter BEQ = 6'd24;
parameter BNE = 6'd25;
parameter SLTI = 6'd26;
parameter SLTIU = 6'd27;
parameter LUI = 6'd28;
parameter J = 6'd29;
parameter JAL = 6'd30;

parameter CLZ = 6'd31;
parameter JALR = 6'd32;
parameter MTHI = 6'd33;
parameter MTLO = 6'd34;
parameter MFHI = 6'd35;
parameter MFLO = 6'd36;
parameter SB = 6'd37;
parameter SH = 6'd38;
parameter LB = 6'd39;
parameter LH = 6'd40;
parameter LBU = 6'd41;

```

parameter LHU      = 6'd42;
parameter ERET     = 6'd43;
parameter BREAK    = 6'd44;
parameter SYSCALL  = 6'd45;
parameter TEQ      = 6'd46;
parameter MFC0     = 6'd47;
parameter MTC0     = 6'd48;
parameter MUL      = 6'd49;
parameter MULTU    = 6'd50;
parameter DIV      = 6'd51;
parameter DIVU     = 6'd52;
parameter BGEZ     = 6'd53;

wire cat_ena;                //是否需要拼接
wire [31:0] a, b;            //ALU 的 A、B 运算输入端
wire [3:0] aluc;             //ALUC 四位运算指令
wire [31:0] alu_data_out;    //ALU 输出的数据
wire zero, carry, negative, overflow; //四个标志位
wire [31:0] Rd_data_in;      //要向寄存器中写入的值
wire [31:0] Rs_data_out;     //Rs 对应的寄存器的输出值
wire [31:0] Rt_data_out;     //Rt 对应的寄存器的输出值
wire [31:0] pc_addr_in;
wire [31:0] pc_addr_out;
wire HI_w;                   //使能信号，是否是向 HI 中写入数
wire LO_w;                   //使能信号，是否是向 LO 中写入数
wire [4:0] ext_ena;
wire [31:0] HI_out;          //从 HI 传出的数
wire [31:0] LO_out;          //从 LO 传出的数
wire sign_flag;              //是否有符号乘法
wire [31:0] DIV_A;           //输入的被除数
wire [31:0] DIV_B;           //输入的除数
/* 连接各模块 */
/* 符号、数据扩展器线路 */
wire [31:0] ext1_out;
wire [31:0] ext5_out;
wire [31:0] ext16_out;
wire signed [31:0] ext16_out_signed;
wire signed [31:0] ext18_out_signed;
wire [31:0] DIV_R;           //余数
wire [31:0] DIV_Q;           //商
wire [31:0] MUL_A;           //输入的乘数 A
wire [31:0] MUL_B;           //输入的乘数 B
wire [31:0] MUL_HI;          //高 32 位结果
wire [31:0] MUL_LO;          //低 32 位结果

```

```

wire [31:0] cp0_out;          //CP0 输出的数据
wire [31:0] epc_out;          //CP0 送出的 PC 地址，用于更新 PC
wire [31:0] CLZ_out;          //CLZ 最终输出内容
assign cat_out = cat_ena ? {pc_out[31:28], address[25:0], 2'h0} : 32'hz;

assign ext18_out_signed=(op_flags[BEQ]||op_flags[BNE]|| op_flags[BGEZ]) ?
{{14{immediate[15]}}, immediate[15:0], 2'b0} : 32'hz;

assign ext16_out= (op_flags[ANDI] || op_flags[ORI] || op_flags[XORI]
|| op_flags[LUI]) ? { 16'h0 , immediate[15:0] } : 32'hz;

assign ext16_out_signed = (op_flags[ADDI] || op_flags[ADDIU] ||
op_flags[LW] || op_flags[SW] || op_flags[SLTI] || op_flags[SLTIU] ||
op_flags[SB] || op_flags[SH] ||op_flags[LB] || op_flags[LH] ||
op_flags[LBU]|| op_flags[LHU]) ?{{16{immediate[15]}}, immediate[15:0] } :
32'hz;

assign ext1_out = (op_flags[SLT] || op_flags[SLTU]) ? negative :
(op_flags[SLTI] || op_flags[SLTIU]) ? carry : 32'hz;

assign ext5_out = (op_flags[SLL] || op_flags[SRL] || op_flags[SRA]) ?
shamt : 32'hz;

/* ||拼接器线路 */
wire [31:0] cat_out;
wire [53:0] op_flags;
wire [4:0] RsC;
wire [4:0] RtC;
wire [4:0] RdC;
wire [4:0] shamt;
wire [15:0] immediate;          //立即数（I 型指令用）
wire [25:0] address;           //跳转地址（J 型指令用）
wire reg_w;                    //RegFile 寄存器堆是否可写入
wire [4:0] cause;              //中断情况
wire [10:0] mux;               //10 个多路选择器的状态
wire [2:0] mux6_1;             //6 路选择器的选择信号
wire [2:0] mux7_1;             //7 路选择器的选择信号

/* 多路选择器线路 */
wire [31:0] mux1_out;
wire [31:0] mux2_out;
wire [31:0] mux3_out;
wire [31:0] mux4_out;
wire [31:0] mux5_out;

```

```

wire [31:0] mux6_out;
wire [31:0] mux7_out;
wire [31:0] mux8_out;
wire [31:0] mux9_out;
wire [31:0] mux10_out;
wire [31:0] mux6_1_out;
wire [31:0] mux7_1_out;
wire [31:0] npc;
wire [31:0] add_out_61;
wire [31:0] add_out_71;
assign npc = pc_addr_out + 4;

assign mux1_out = mux[1] ? Rt_data_out : Rs_data_out;
assign mux2_out = mux[2] ? ext16_out_signed : ext16_out;
assign mux3_out = mux[3] ? ext5_out : Rs_data_out;
assign mux4_out = mux[4] ? mux2_out : Rt_data_out;
assign mux5_out = mux[5] ? mux9_out : Rs_data_out;
assign mux6_out = mux[6] ? mux10_out : Rs_data_out;
assign mux7_out = mux[7] ? LO_out : HI_out;
assign mux8_out = mux[8] ? alu_data_out : ext1_out;
assign mux9_out = mux[9] ? DIV_R : MUL_HI;
assign mux10_out = mux[10] ? DIV_Q : MUL_LO;

assign sb_flag = op_flags[SB];
assign sh_flag = op_flags[SH];
assign sw_flag = op_flags[SW];
assign lb_flag = op_flags[LB];
assign lh_flag = op_flags[LH];
assign lbu_flag = op_flags[LBU];
assign lhu_flag = op_flags[LHU];
assign lw_flag = op_flags[LW];
assign a = mux3_out;
assign b = op_flags[BGEZ] ? 32'd0 : mux4_out;
assign pc_out = pc_addr_out;
assign pc_addr_in = mux6_1_out;
assign add_out_61 = ext18_out_signed + npc;
assign add_out_71 = pc_addr_out + 4;
assign dm_data_w = Rt_data_out;
assign dm_ena = (dm_r || dm_w) ? 1'b1 : 1'b0;
assign dm_addr = alu_data_out;

assign Rd_data_in = mux7_1_out;
assign sign_flag = op_flags[MUL] || op_flags[DIV] ? 1'b1 : 1'b0;
assign MUL_A = op_flags[MUL] || op_flags[MULTU] ? Rs_data_out : 32'hz;

```

```

assign MUL_B = op_flags[MUL] || op_flags[MULTU] ? Rt_data_out : 32'hz;
assign DIV_A = op_flags[DIV] || op_flags[DIVU] ? Rs_data_out : 32'hz;
assign DIV_B = op_flags[DIV] || op_flags[DIVU] ? Rt_data_out : 32'hz;

```

/* 实例化译码器 */

```

Decoder Decoder_inst(
    .instr_in(instr_in),
    .op_flags(op_flags),
    .RsC(RsC),
    .RtC(RtC),
    .RdC(RdC),
    .shamt(shamt),
    .immediate(immediate),
    .address(address)
);

```

/* 实例化控制器 */

```

Controler Controler_inst(
    .op_flags(op_flags),
    .zero_flag(zero),
    .sign_flag(negative),
    .reg_w(reg_w),
    .aluc(aluc),
    .dm_r(dm_r),
    .dm_w(dm_w),
    .HI_w(HI_w),
    .LO_w(LO_w),
    .cause(cause),
    .ext_ena(ext_ena),
    .cat_ena(cat_ena),
    .mux(mux),
    .mux6_1(mux6_1),
    .mux7_1(mux7_1)
);

```

/* 实例化 ALU */

```

ALU ALU_inst(
    .A(a),                //对应 A 接口
    .B(b),                //对应 B 接口
    .ALUC(aluc),          //ALUC 四位操作指令
    .alu_data_out(alu_data_out), //输出数据
    .zero(zero),
    .carry(carry),
    .negative(negative),

```

```

        .overflow(overflow)
    );

/* 实例化寄存器堆 */
regfile cpu_ref(
    .reg_clk(clk),
    .reg_ena(ena),
    .rst(rst),
    .reg_w(reg_w),
    .RdC(RdC),
    .RtC(RtC),
    .RsC(RsC),
    .Rd_data_in(Rd_data_in),
    .Rs_data_out(Rs_data_out),
    .Rt_data_out(Rt_data_out)
);

/* 实例化 PC 寄存器 */
PC PC_inst(
    .pc_clk(clk),
    .pc_ena(ena),
    .rst(rst),
    .pc_addr_in(pc_addr_in),
    .pc_addr_out(pc_addr_out)
);

/* 实例化 HI_LO 寄存器 */
HI_LO HI_LO_inst(
    .HI_LO_clk(clk),
    .HI_LO_ena(ena),
    .HI_LO_rst(rst),
    .HI_in(mux5_out),
    .LO_in(mux6_out),
    .HI_w(HI_w),
    .LO_w(LO_w),
    .HI_out(HI_out),
    .LO_out(LO_out)
);

MUX MUX6_1(
    .chosen(mux6_1),          //8 选 1, 用三位, 下面表示第几根线
    .line0(Rs_data_out),
    .line1(npc),
    .line2(add_out_61),

```

```

        .line3(32'h00000004),
        .line4(epc_out),
        .line5(cat_out),
        .line6(32'bz),
        .line7(32'bz),
        .MUX_out(mux6_1_out)
    );

MUX MUX7_1(
    .chosen(mux7_1),
    .line0(dm_data),
    .line1(mux7_out),
    .line2(CLZ_out),
    .line3(mux8_out),
    .line4(MUL_L0),
    .line5(add_out_71),
    .line6(cp0_out),
    .line7(32'bz),
    .MUX_out(mux7_1_out)    //选择的线
);

DIV DIV_inst(
    .sign_flag(sign_flag), //是否有符号除法
    .A(DIV_A),              //输入的被除数
    .B(DIV_B),              //输入的除数
    .R(DIV_R),              //余数
    .Q(DIV_Q)               //商
);

MUL MUL_inst(
    .sign_flag(sign_flag), //是否有符号乘法
    .A(MUL_A),              //输入的乘数 A
    .B(MUL_B),              //输入的乘数 B
    .HI(MUL_HI),            //高 32 位结果
    .LO(MUL_LO)             //低 32 位结果
);

CP0 CP0_inst(
    .cp0_clk(clk),          //CP0 的时钟信号
    .cp0_rst(rst),          //CP0 寄存器复位信号
    .cp0_ena(ena),          //CP0 的使能信号
    .MFC0(op_flags[MFC0]),  //此时 CP0 要执行的指令是否为 MFC0
    .MTC0(op_flags[MTC0]),  //此时 CP0 要执行的指令是否为 MTC0
    .ERET(op_flags[ERET]),  //此时 CP0 要执行的指令是否为 ERET

```



```

        .PC(pc_addr_out),          //要写入 CP0 的 PC 地址
        .addr(mux1_out),          //要写入的寄存器地址
        .cause(cause),           //当前中断类型
        .data_in(Rt_data_out),    //从 GR 中传入的数据，用于写入 CP0
        .CP0_out(cp0_out),        //CP0 输出的数据
        .EPC_out(epc_out)         //CP0 送出的 PC 地址，用于更新 PC
    );

    CLZ CLZ_inst(
        .CLZ_in(Rs_data_out),      //要计算前导 0 的数值
        .CLZ_out(CLZ_out)          //输出前导 0 的个数
    );

Endmodule

Decoder 模块

module Decoder(
    input  [31:0] instr_in,        //需要译码的指令，也就是当前要执行的指令
    output [53:0] op_flags,        //54 条指令对应的标志位
    output [4:0]  RsC,             //Rs 对应的寄存器的地址
    output [4:0]  RtC,             //Rt 对应的寄存器的地址
    output [4:0]  RdC,             //Rd 对应的寄存器的地址
    output [4:0]  shamt,           //位移偏移量
    output [15:0] immediate,       //立即数
    output [25:0] address          //跳转地址（J 型指令用）
);
/* 定义各指令在原指令中对应的编码 */
parameter ADD_OPE   = 6'b100000;
parameter ADDU_OPE  = 6'b100001;
parameter SUB_OPE   = 6'b100010;
parameter SUBU_OPE  = 6'b100011;
parameter AND_OPE   = 6'b100100;
parameter OR_OPE    = 6'b100101;
parameter XOR_OPE   = 6'b100110;
parameter NOR_OPE   = 6'b100111;
parameter SLT_OPE   = 6'b101010;
parameter SLTU_OPE  = 6'b101011;
parameter SLL_OPE   = 6'b000000;
parameter SRL_OPE   = 6'b000010;
parameter SRA_OPE   = 6'b000011;
parameter SLLV_OPE  = 6'b000100;
parameter SRLV_OPE  = 6'b000110;

```

```

parameter SRAV_OPE = 6'b000111;
parameter JR_OPE   = 6'b001000;
parameter ADDI_OPE = 6'b001000;
parameter ADDIU_OPE = 6'b001001;
parameter ANDI_OPE = 6'b001100;
parameter ORI_OPE  = 6'b001101;
parameter XORI_OPE = 6'b001110;
parameter LW_OPE   = 6'b100011;
parameter SW_OPE   = 6'b101011;
parameter BEQ_OPE  = 6'b000100;
parameter BNE_OPE  = 6'b000101;
parameter SLTI_OPE = 6'b001010;
parameter SLTIU_OPE = 6'b001011;
parameter LUI_OPE  = 6'b001111;
parameter J_OPE    = 6'b000010;
parameter JAL_OPE  = 6'b000011;

/* 54 条指令添加部分 */
parameter CLZ_OPE   = 6'b100000;
parameter JALR_OPE  = 6'b001001;
parameter MTHI_OPE  = 6'b010001;
parameter MFHI_OPE  = 6'b010000;
parameter MTLO_OPE  = 6'b010011;
parameter MFLO_OPE  = 6'b010010;
parameter SB_OPE    = 6'b101000;
parameter SH_OPE    = 6'b101001;
parameter LB_OPE    = 6'b100000;
parameter LH_OPE    = 6'b100001;
parameter LBU_OPE   = 6'b100100;
parameter LHU_OPE   = 6'b100101;
parameter ERET_OPE  = 6'b011000;
parameter BREAK_OPE = 6'b001101;
parameter SYSCALL_OPE = 6'b001100;
parameter TEQ_OPE   = 6'b110100;
parameter MFC0_OPE  = 5'b000000;
parameter MTC0_OPE  = 5'b00100;
parameter MUL_OPE   = 6'b000010;
parameter MULTU_OPE = 6'b011001;
parameter DIV_OPE   = 6'b011010;
parameter DIVU_OPE  = 6'b011011;
parameter BGEZ_OPE  = 6'b000001;

/* 54 条指令对应的标志位 */
parameter ADD = 6'd0;

```

parameter ADDU = 6'd1;
parameter SUB = 6'd2;
parameter SUBU = 6'd3;
parameter AND = 6'd4;
parameter OR = 6'd5;
parameter XOR = 6'd6;
parameter NOR = 6'd7;
parameter SLT = 6'd8;
parameter SLTU = 6'd9;
parameter SLL = 6'd10;
parameter SRL = 6'd11;
parameter SRA = 6'd12;
parameter SLLV = 6'd13;
parameter SRLV = 6'd14;
parameter SRAV = 6'd15;
parameter JR = 6'd16;
parameter ADDI = 6'd17;
parameter ADDIU = 6'd18;
parameter ANDI = 6'd19;
parameter ORI = 6'd20;
parameter XORI = 6'd21;
parameter LW = 6'd22;
parameter SW = 6'd23;
parameter BEQ = 6'd24;
parameter BNE = 6'd25;
parameter SLTI = 6'd26;
parameter SLTIU = 6'd27;
parameter LUI = 6'd28;
parameter J = 6'd29;
parameter JAL = 6'd30;

parameter CLZ = 6'd31;
parameter JALR = 6'd32;
parameter MTHI = 6'd33;
parameter MTLO = 6'd34;
parameter MFHI = 6'd35;
parameter MFLO = 6'd36;
parameter SB = 6'd37;
parameter SH = 6'd38;
parameter LB = 6'd39;
parameter LH = 6'd40;
parameter LBU = 6'd41;
parameter LHU = 6'd42;
parameter ERET = 6'd43;

```

parameter BREAK    = 6'd44;
parameter SYSCALL  = 6'd45;
parameter TEQ      = 6'd46;
parameter MFC0     = 6'd47;
parameter MTC0     = 6'd48;
parameter MUL      = 6'd49;
parameter MULTU    = 6'd50;
parameter DIV      = 6'd51;
parameter DIVU     = 6'd52;
parameter BGEZ     = 6'd53;

```

```

/* 下面是赋值 */

```

```

/* 对指令进行译码，判断是哪个指令 */

```

```

assign op_flags[ADD]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
ADD_OPE )) ? 1'b1 : 1'b0;
assign op_flags[ADDU]  = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
ADDU_OPE)) ? 1'b1 : 1'b0;
assign op_flags[SUB]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SUB_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SUBU]  = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SUBU_OPE)) ? 1'b1 : 1'b0;
assign op_flags[AND]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
AND_OPE )) ? 1'b1 : 1'b0;
assign op_flags[OR]    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
OR_OPE )) ? 1'b1 : 1'b0;
assign op_flags[XOR]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
XOR_OPE )) ? 1'b1 : 1'b0;
assign op_flags[NOR]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
NOR_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SLT]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLT_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SLTU]  = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLTU_OPE)) ? 1'b1 : 1'b0;
assign op_flags[SLL]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLL_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SRL]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SRL_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SRA]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SRA_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SLLV]  = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLLV_OPE)) ? 1'b1 : 1'b0;
assign op_flags[SRLV]  = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SRLV_OPE)) ? 1'b1 : 1'b0;
assign op_flags[RAV]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==

```

```

SRAV_OPE)) ? 1'b1 : 1'b0;
assign op_flags[JIR]    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
JR_OPE )) ? 1'b1 : 1'b0;
assign op_flags[ADDI]   = (instr_in[31:26] == ADDI_OPE ) ? 1'b1 : 1'b0;
assign op_flags[ADDIU]  = (instr_in[31:26] == ADDIU_OPE) ? 1'b1 : 1'b0;
assign op_flags[ANDI]   = (instr_in[31:26] == ANDI_OPE ) ? 1'b1 : 1'b0;
assign op_flags[ORI]    = (instr_in[31:26] == ORI_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[XORI]   = (instr_in[31:26] == XORI_OPE ) ? 1'b1 : 1'b0;
assign op_flags[LW]     = (instr_in[31:26] == LW_OPE   ) ? 1'b1 : 1'b0;
assign op_flags[SW]     = (instr_in[31:26] == SW_OPE   ) ? 1'b1 : 1'b0;
assign op_flags[BEQ]    = (instr_in[31:26] == BEQ_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[BNE]    = (instr_in[31:26] == BNE_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[SLTI]   = (instr_in[31:26] == SLTI_OPE ) ? 1'b1 : 1'b0;
assign op_flags[SLTIU]  = (instr_in[31:26] == SLTIU_OPE) ? 1'b1 : 1'b0;
assign op_flags[LUI]    = (instr_in[31:26] == LUI_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[J]      = (instr_in[31:26] == J_OPE    ) ? 1'b1 : 1'b0;
assign op_flags[JAL]    = (instr_in[31:26] == JAL_OPE  ) ? 1'b1 : 1'b0;

assign op_flags[CLZ]    = ((instr_in[31:26] == 6'b011100) &&
(instr_in[5:0] == CLZ_OPE )) ? 1'b1 : 1'b0;
assign op_flags[JALR]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== JALR_OPE )) ? 1'b1 : 1'b0;
assign op_flags[MTHI]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== MTHI_OPE )) ? 1'b1 : 1'b0;
assign op_flags[MTLO]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== MTLO_OPE )) ? 1'b1 : 1'b0;
assign op_flags[MFHI]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== MFHI_OPE )) ? 1'b1 : 1'b0;
assign op_flags[MFLO]   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== MFLO_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SB]     = (instr_in[31:26] == SB_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[SH]     = (instr_in[31:26] == SH_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[LB]     = (instr_in[31:26] == LB_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[LH]     = (instr_in[31:26] == LH_OPE  ) ? 1'b1 : 1'b0;
assign op_flags[LBU]    = (instr_in[31:26] == LBU_OPE ) ? 1'b1 : 1'b0;
assign op_flags[LHU]    = (instr_in[31:26] == LHU_OPE ) ? 1'b1 : 1'b0;
assign op_flags[ERET]   = ((instr_in[31:26] == 6'b010000) &&
(instr_in[5:0] == ERET_OPE )) ? 1'b1 : 1'b0;
assign op_flags[BREAK]  = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== BREAK_OPE )) ? 1'b1 : 1'b0;
assign op_flags[SYSCALL] = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== SYSCALL_OPE )) ? 1'b1 : 1'b0;
assign op_flags[TEQ]    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== TEQ_OPE )) ? 1'b1 : 1'b0;

```

```

assign op_flags[MFC0]      = ((instr_in[31:26] == 6'b010000) &&
(instr_in[5:0] == 6'h0) && (instr_in[25:21] == MFC0_OPE)) ? 1'b1 : 1'b0;
assign op_flags[MTC0]      = ((instr_in[31:26] == 6'b010000) &&
(instr_in[5:0] == 6'h0) && (instr_in[25:21] == MTC0_OPE)) ? 1'b1 : 1'b0;
assign op_flags[MUL]       = ((instr_in[31:26] == 6'b011100) &&
(instr_in[5:0] == MUL_OPE    )) ? 1'b1 : 1'b0;
assign op_flags[MULTU]     = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== MULTU_OPE    )) ? 1'b1 : 1'b0;
assign op_flags[DIV]       = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== DIV_OPE      )) ? 1'b1 : 1'b0;
assign op_flags[DIVU]      = ((instr_in[31:26] == 6'h0) && (instr_in[5:0]
== DIVU_OPE     )) ? 1'b1 : 1'b0;
assign op_flags[BGEZ]      = ((instr_in[31:26] == BGEZ_OPE) &&
(instr_in[20:16] == 5'b00001) ) ? 1'b1 : 1'b0;

```

/* 取出指令中各部分的值 */

```

assign RsC = (op_flags[ADD]   || op_flags[ADDU] || op_flags[SUB]   ||
op_flags[SUBU] || op_flags[AND] || op_flags[OR]   || op_flags[XOR] ||
op_flags[NOR]   || op_flags[SLT] || op_flags[SLTU] || op_flags[SLLV] ||
op_flags[SRLV]  || op_flags[SRAV] || op_flags[JR]   || op_flags[ADDI] ||
op_flags[ADDIU] || op_flags[ANDI] || op_flags[ORI]   || op_flags[XORI] ||
op_flags[LW]    || op_flags[SW]   || op_flags[BEQ]   || op_flags[BNE] ||
op_flags[SLTI]  || op_flags[SLTIU] || op_flags[CLZ]   || op_flags[JALR] ||
op_flags[MTHI]  || op_flags[MTLO] || op_flags[SB]    || op_flags[SH]   ||
op_flags[LB]    || op_flags[LH]   || op_flags[LBU]   || op_flags[LHU] ||
op_flags[TEQ]   || op_flags[MUL]  || op_flags[MULTU] || op_flags[DIV] ||
op_flags[DIVU]  || op_flags[BGEZ]) ? instr_in[25:21] : (op_flags[MTC0] ?
instr_in[15:11] : 5'hz);

```

```

assign RtC = (op_flags[ADD]   || op_flags[ADDU]   || op_flags[SUB]
|| op_flags[SUBU] || op_flags[AND] || op_flags[OR]   || op_flags[XOR]
|| op_flags[NOR]   || op_flags[SLT] || op_flags[SLTU] || op_flags[SLL]
|| op_flags[SRL]   || op_flags[SRA] || op_flags[SLLV] || op_flags[SRLV]
|| op_flags[SRAV] || op_flags[SW]   || op_flags[BEQ]   || op_flags[BNE]
|| op_flags[SB]    || op_flags[SH]   || op_flags[TEQ]   || op_flags[MTC0]
|| op_flags[MUL]    || op_flags[MULTU] || op_flags[DIV]
|| op_flags[DIVU]) ? instr_in[20:16] : (op_flags[MFC0] ? instr_in[15:11] :
5'hz);

```

```

assign RdC = (op_flags[ADD]   || op_flags[ADDU]   || op_flags[SUB]   ||
op_flags[SUBU] || op_flags[AND] || op_flags[OR]   || op_flags[XOR] ||
op_flags[NOR]   || op_flags[SLT] || op_flags[SLTU] || op_flags[SLL] ||
op_flags[SRL]   || op_flags[SRA] || op_flags[SLLV] || op_flags[SRLV] ||

```

```

op_flags[SRAV] || op_flags[CLZ] || op_flags[JALR] || op_flags[MFHI] ||
op_flags[MFLO] || op_flags[MUL]) ? instr_in[15:11] : ((op_flags[ADDI] ||
op_flags[ADDIU] || op_flags[ANDI] || op_flags[ORI] || op_flags[XORI]
|| op_flags[LW] || op_flags[SLTI] || op_flags[SLTIU] ||
op_flags[LUI] || op_flags[MFC0] || op_flags[LB] || op_flags[LH] ||
op_flags[LBU] || op_flags[LHU] ) ? instr_in[20:16] : (op_flags[JAL] ?
5'd31 : 5'hz));

```

```

assign shamt = (op_flags[SLL] || op_flags[SRL] || op_flags[SRA]) ?
instr_in[10:6] : 5'hz;

```

```

assign immediate = (op_flags[ADDI] || op_flags[ADDIU] || op_flags[ANDI]
|| op_flags[ORI] || op_flags[XORI] || op_flags[LW] || op_flags[SW]
|| op_flags[BEQ] || op_flags[BNE] || op_flags[SLTI] || op_flags[SLTIU]
|| op_flags[LUI] || op_flags[SB] || op_flags[SH] || op_flags[LB]
|| op_flags[LH] || op_flags[LBU] || op_flags[LHU] || op_flags[BGEZ]) ?
instr_in[15:0] : 16'hz;

```

```

assign address = (op_flags[J] || op_flags[JAL]) ? instr_in[25:0] : 26'hz;

```

```

endmodule

```

Controller 模块

```

module Controller(
    input [53:0] op_flags,
    input zero_flag,
    input sign_flag,
    output reg_w,
    output [3:0] aluc,
    output dm_r,
    output dm_w,
    output HI_w,
    output LO_w,
    output [4:0] cause,
    output [4:0] ext_ena,
    output cat_ena,
    output [10:0] mux,
    output [2:0] mux6_1,
    output [2:0] mux7_1
);

```

```

/* 54 条指令的状态编码 */

```

parameter ADD = 6'd0;
parameter ADDU = 6'd1;
parameter SUB = 6'd2;
parameter SUBU = 6'd3;
parameter AND = 6'd4;
parameter OR = 6'd5;
parameter XOR = 6'd6;
parameter NOR = 6'd7;
parameter SLT = 6'd8;
parameter SLTU = 6'd9;
parameter SLL = 6'd10;
parameter SRL = 6'd11;
parameter SRA = 6'd12;
parameter SLLV = 6'd13;
parameter SRLV = 6'd14;
parameter SRAV = 6'd15;
parameter JR = 6'd16;
parameter ADDI = 6'd17;
parameter ADDIU = 6'd18;
parameter ANDI = 6'd19;
parameter ORI = 6'd20;
parameter XORI = 6'd21;
parameter LW = 6'd22;
parameter SW = 6'd23;
parameter BEQ = 6'd24;
parameter BNE = 6'd25;
parameter SLTI = 6'd26;
parameter SLTIU = 6'd27;
parameter LUI = 6'd28;
parameter J = 6'd29;
parameter JAL = 6'd30;

parameter CLZ = 6'd31;
parameter JALR = 6'd32;
parameter MTHI = 6'd33;
parameter MTLO = 6'd34;
parameter MFHI = 6'd35;
parameter MFLO = 6'd36;
parameter SB = 6'd37;
parameter SH = 6'd38;
parameter LB = 6'd39;
parameter LH = 6'd40;
parameter LBU = 6'd41;
parameter LHU = 6'd42;


```

parameter ERET      = 6'd43;
parameter BREAK     = 6'd44;
parameter SYSCALL   = 6'd45;
parameter TEQ       = 6'd46;
parameter MFC0      = 6'd47;
parameter MTC0      = 6'd48;
parameter MUL       = 6'd49;
parameter MULTU     = 6'd50;
parameter DIV       = 6'd51;
parameter DIVU      = 6'd52;
parameter BGEZ      = 6'd53;

```

/* 下面是赋值，也就是根据要执行的操作决定各元器件的状态 */

```

assign reg_w = (!op_flags[Jr]    && !op_flags[Sw]    && !op_flags[BEQ]
&& !op_flags[BNE]&& !op_flags[J]  &&!op_flags[MTHI]  && !op_flags[MTLO]
&& !op_flags[SB] && !op_flags[SH]  && !op_flags[ERET] &&!op_flags[BREAK]
&& !op_flags[SYSCALL] && !op_flags[TEQ]  && !op_flags[MTC0]
&&!op_flags[MULTU]&&!op_flags[DIV]&&!op_flags[DIVU] && !op_flags[BGEZ]) ?
1'b1 : 1'b0;

```

```

assign aluc[3] = (op_flags[SLT]  || op_flags[SLTU]  || op_flags[SLLV] ||
op_flags[SRLV] ||op_flags[SRAV] || op_flags[SLL]  || op_flags[SRL]  ||
op_flags[SRA]  || op_flags[SLTI] || op_flags[SLTIU] || op_flags[LUI]) ?
1'b1 : 1'b0;

```

```

assign aluc[2] = (op_flags[AND]  || op_flags[OR]    || op_flags[XOR]  ||
op_flags[NOR]  ||op_flags[SLLV] || op_flags[SRLV]  || op_flags[SRAV] ||
op_flags[SLL]  ||op_flags[SRL]  || op_flags[SRA]    || op_flags[ANDI] ||
op_flags[ORI]  ||op_flags[XORI]) ? 1'b1 : 1'b0;

```

```

assign aluc[1] = (op_flags[ADD]  || op_flags[SUB]    || op_flags[XOR]  ||
op_flags[NOR]  ||op_flags[SLT]  || op_flags[SLTU]  || op_flags[SLLV] ||
op_flags[SLL]  ||op_flags[ADDI] || op_flags[XORI]  || op_flags[SLTI] ||
op_flags[SLTIU]||op_flags[SB]    || op_flags[SH]    || op_flags[LB]    ||
op_flags[LH]   ||op_flags[LBU]  || op_flags[LHU]   || op_flags[TEQ]  ||
op_flags[BGEZ]) ? 1'b1 : 1'b0;

```

```

assign aluc[0] = (op_flags[SUB]  || op_flags[SUBU]  || op_flags[OR]    ||
op_flags[NOR]  ||op_flags[SLT]  || op_flags[SLLV]  || op_flags[SRLV] ||
op_flags[SLL]  ||op_flags[SRL]  || op_flags[ORI]    || op_flags[SLTI] ||
op_flags[LUI]  ||op_flags[BEQ]  || op_flags[BNE]    || op_flags[MFC0] ||
op_flags[BGEZ]) ? 1'b1 : 1'b0;

```

```

assign dm_r = op_flags[LW] || op_flags[LB] || op_flags[LH] || op_flags[LBU]
|| op_flags[LHU] ? 1'b1 : 1'b0;

```

```

assign dm_w = op_flags[Sw] || op_flags[SB] || op_flags[SH] ? 1'b1 : 1'b0;

```

```

assign HI_w = (op_flags[MTHI] || op_flags[MULTU] || op_flags[DIV] ||
op_flags[DIVU]) ? 1'b1 : 1'b0;
assign LO_w = (op_flags[MTLO] || op_flags[MULTU] || op_flags[DIV] ||
op_flags[DIVU]) ? 1'b1 : 1'b0;

assign cause = op_flags[SYSCALL] ? 5'b01000 : (op_flags[BREAK] ? 5'b01001 :
(op_flags[TEQ] ? 5'b01101 : 5'bz));

assign ext_ena[4] = (op_flags[BEQ] || op_flags[BNE] ||
op_flags[BGEZ]) ? 1'b1 : 1'b0; //EXT18(S)
assign ext_ena[3] = (op_flags[ADDI] || op_flags[ADDIU] || op_flags[LW]
|| op_flags[SW] || op_flags[SLTI] || op_flags[SLTIU] || op_flags[SB] ||
op_flags[SH] || op_flags[LB] || op_flags[LH] || op_flags[LBU] ||
op_flags[LHU]) ? 1'b1 : 1'b0; //EXT16(S)

assign ext_ena[2] = (op_flags[ANDI] || op_flags[ORI] || op_flags[XORI]
|| op_flags[LUI]) ? 1'b1 : 1'b0; //EXT16
assign ext_ena[1] = (op_flags[SLL] || op_flags[SRL] || op_flags[SRA]) ?
1'b1 : 1'b0; //EXT5
assign ext_ena[0] = (op_flags[SLT] || op_flags[SLTU] || op_flags[SLTI]
|| op_flags[SLTIU]) ? 1'b1 : 1'b0; //EXT1

assign cat_ena = (op_flags[J] || op_flags[JAL]) ? 1'b1 : 1'b0;

assign mux[1] = op_flags[MFC0] ? 1'b1 : 1'b0;
assign mux[2] = (op_flags[ANDI] || op_flags[ORI] || op_flags[XORI] ||
op_flags[LUI]) ? 1'b0 : 1'b1; //注意这里是选0
assign mux[3] = (op_flags[SLL] || op_flags[SRL] || op_flags[SRA]) ?
1'b1 : 1'b0;
assign mux[4] = (op_flags[ADDI] || op_flags[ADDIU] || op_flags[ANDI] ||
op_flags[ORI] || op_flags[XORI] || op_flags[LW] || op_flags[SW] ||
op_flags[SLTI] || op_flags[SLTIU] || op_flags[LUI] || op_flags[SB] ||
op_flags[SH] || op_flags[LB] || op_flags[LH] || op_flags[LBU] ||
op_flags[LHU]) ? 1'b1 : 1'b0;
assign mux[5] = op_flags[MTHI] ? 1'b0 : 1'b1; //注意这里是选0
assign mux[6] = op_flags[MTLO] ? 1'b0 : 1'b1; //注意这里是选0
assign mux[7] = op_flags[MFHI] ? 1'b0 : 1'b1; //注意这里是选0
assign mux[8] = (op_flags[SLT] || op_flags[SLTU] || op_flags[SLTI] ||
op_flags[SLTIU]) ? 1'b0 : 1'b1; //注意这里是选0
assign mux[9] = op_flags[MULTU] ? 1'b0 : 1'b1;
assign mux[10] = op_flags[MULTU] ? 1'b0 : 1'b1;

assign mux6_1 = (op_flags[J] || op_flags[JAL]) ? 3'b000 : ((op_flags[BEQ]

```

```

&& zero_flag) || (op_flags[BNE] && !zero_flag) || (op_flags[BGEZ]
&& !sign_flag) ? 3'b010 : (op_flags[BREAK] || op_flags[SYSCALL] ||
(op_flags[TEQ] && zero_flag) ? 3'b011 :
(op_flags[ERET]?3'b100:(op_flags[J]|op_flags[JAL] ? 3'b101 : 3'b001))));

assign mux7_1 = (op_flags[LW] || op_flags[LB] || op_flags[LH] ||
op_flags[LBU] || op_flags[LHU]) ? 3'b000 : (op_flags[MFHI] ||
op_flags[MFL0]?3'b001:(op_flags[CLZ]?3'b010:(op_flags[MUL]?3'b100:(op_f
lags[JALR]||op_flags[JAL]?3'b101:(op_flags[MFC0] ? 3'b110 : 3'b011))));
endmodule

```

ALU 模块，算术逻辑单元模块，实现多种算术、逻辑运算

```

module ALU(
    input [31:0] A,           //对应 A 接口
    input [31:0] B,           //对应 B 接口
    input [3:0] ALUC,         //ALUC 四位操作指令
    output [31:0] alu_data_out, //输出数据
    output zero,              //ZF 标志位，BEQ/BNE 使用
    output carry,             //CF 标志位，SLTI/SLTIU 使用
    output negative,          //NF(SF)标志位，SLT/SLTU 使用
    output overflow
);
/* 定义各指令对应的操作 */
parameter ADDU = 4'b0000;
parameter ADD = 4'b0010;
parameter SUBU = 4'b0001;
parameter SUB = 4'b0011;
parameter AND = 4'b0100;
parameter OR = 4'b0101;
parameter XOR = 4'b0110;
parameter NOR = 4'b0111;
parameter LUI1 = 4'b1000;
parameter LUI2 = 4'b1001;
parameter SLT = 4'b1011;
parameter SLTU = 4'b1010;
parameter SRA = 4'b1100;
parameter SLL = 4'b1110;
parameter SLA = 4'b1111;
parameter SRL = 4'b1101;
/* 定义一些内部用的变量 */
reg [32:0] result;
wire signed [31:0] signedA,signedB;

```

```

assign signedA = A;
assign signedB = B;

always @(*)
begin
    case(ALUC)
        ADDU:      begin result <= A + B;                end
        ADD:       begin result <= signedA + signedB;    end
        SUBU:      begin result <= A - B;                end
        SUB:       begin result <= signedA - signedB;    end
        AND:       begin result <= A & B;                end
        OR:        begin result <= A | B;                end
        XOR:       begin result <= A ^ B;                end
        NOR:       begin result <= ~(A | B);             end
        LUI1,LUI2: begin result <= { B[15:0] , 16'b0 };    end
        SLT:       begin result <= signedA - signedB;    end
        SLTU:      begin result <= A - B;                end
        SRA:       begin result <= signedB >>> signedA;  end
        SLL,SLA:   begin result <= B << A;                end
        SRL:       begin result <= B >> A;                end
    endcase
end

assign alu_data_out = result[31:0];
assign zero = (result == 32'b0) ? 1 : 0;
assign carry = result[32];
assign negative = (ALUC == SLT || ALUC == SUB ? (signedA < signedB) :
((ALUC == SLTU) ? (A < B) : 1'b0));
assign overflow = result[32];

endmodule

```

regfile 模块，寄存器堆，含有 32 个寄存器读写内容

```

module regfile(
    input reg_clk,
    input reg_ena,
    input rst,
    input reg_w,
    input [4:0] RdC,
    input [4:0] RtC,
    input [4:0] RsC,
    input [31:0] Rd_data_in,

```

```

        output [31:0] Rs_data_out,
        output [31:0] Rt_data_out
    );

    // 定义寄存器堆
    reg [31:0] array_reg[31:0];
    integer i;
    // 异步读取操作
    assign Rs_data_out = (reg_ena) ? array_reg[RsC] : 32'b0;
    assign Rt_data_out = (reg_ena) ? array_reg[RtC] : 32'b0;

    // 异步写入操作
    always @(negedge reg_clk or posedge rst) begin
        if (rst && reg_ena) begin
            // 清空所有寄存器

            for (i = 0; i < 32; i = i + 1) begin
                array_reg[i] <= 32'b0;
            end
            end else if (reg_ena && reg_w && (RdC != 5'd0)) begin
                // 写入数据到指定寄存器, 排除 0 号寄存器
                array_reg[RdC] <= Rd_data_in;
            end
        end
    end

endmodule

```

PC 模块//采用异步读取同步写入方式声明一个 pc_reg[31:0], 为了与 MARS 中指令存储初始地址一致, 让 PC 的初值是 32'h00400000

```

module PC(
    input  pc_clk,
    input  pc_ena,
    input  rst,
    input  [31:0] pc_addr_in,
    output [31:0] pc_addr_out  /
);
    reg [31:0] pc_reg = 32'h00400000; //起始位置在 32'h00400000
    assign pc_addr_out = pc_ena ? pc_reg : 32'hz;

    always @(negedge pc_clk or posedge rst) //复位信号上升沿或时钟下降沿有效
    begin
        if(rst && pc_ena)

```

```

        pc_reg <= 32'h00400000;
    else if(pc_ena)
        pc_reg <= pc_addr_in;

end

endmodule

```

HI_LO 模块，实现对两个 32 位寄存器的读写控制

```

module HI_LO(
    input HI_LO_clk,        //时钟信号
    input HI_LO_ena,        //使能信号
    input HI_LO_rst,        //复位信号
    input [31:0] HI_in,     //向 HI 输入的数
    input [31:0] LO_in,     //向 LO 输入的数
    input HI_w,             //使能信号，是否是向 HI 中写入数
    input LO_w,             //使能信号，是否是向 LO 中写入数
    output [31:0] HI_out,   //从 HI 传出的数
    output [31:0] LO_out    //从 LO 传出的数
);

reg [31:0] HI = 32'd0;     //存储高 32 位数
reg [31:0] LO = 32'd0;     //存储低 32 位数

assign HI_out = HI_LO_ena ? HI : 32'bz;
assign LO_out = HI_LO_ena ? LO : 32'bz;

always @(posedge HI_LO_rst or negedge HI_LO_clk) begin
    if (HI_LO_ena && HI_LO_rst) begin
        HI <= 32'd0;
        LO <= 32'd0;
    end
    else if(HI_LO_ena)
    begin
        if(HI_w)
            HI <= HI_in;
        if(LO_w)
            LO <= LO_in;
    end
end

endmodule

```

MUX, 8 选 1 选择器, 根据不同的选择信号 chosen 选择不同的线

```
module MUX(  
    input [2:0] chosen,      //8 选 1, 用三位  
    input [31:0] line0,     //第一根线  
    input [31:0] line1,     //第二根线  
    input [31:0] line2,     //第三根线  
    input [31:0] line3,     //第四根线  
    input [31:0] line4,     //第五根线  
    input [31:0] line5,     //第六根线  
    input [31:0] line6,     //第七根线  
    input [31:0] line7,     //第八根线  
    output [31:0] MUX_out   //选择的线  
);  
  
assign MUX_out = (chosen == 3'b000 ? line0 :  
    (chosen == 3'b001 ? line1 :  
    (chosen == 3'b010 ? line2 :  
    (chosen == 3'b011 ? line3 :  
    (chosen == 3'b100 ? line4 :  
    (chosen == 3'b101 ? line5 :  
    (chosen == 3'b110 ? line6 :  
    (chosen == 3'b111 ? line7 : 32'bz))))))));  
  
Endmodule
```

DIV, 除法器, 能进行有符号或无符号除法运算, 并且输出余数和商

```
module DIV(  
    input sign_flag,      //是否有符号除法  
    input [31:0] A,       //输入的被除数  
    input [31:0] B,       //输入的除数  
    output [31:0] R,      //余数  
    output [31:0] Q       //商  
);  
  
/* 内部用变量 */  
wire signed [31:0] signed_A; //有符号被除数  
wire signed [31:0] signed_B; //有符号除数  
  
assign signed_A = A;  
assign signed_B = B;
```

```

assign R = (B == 32'd0) ? 32'd0: (sign_flag ? (signed_A % signed_B): (A %
B));
assign Q = (B == 32'd0) ? 32'd0: (sign_flag ? (signed_A / signed_B): (A /
B));
endmodule

```

MUL，乘法器，可以有符号或无符号乘法运算

```

module MUL(
    input sign_flag,    // 是否有符号乘法
    input [31:0] A,     // 输入的乘数 A
    input [31:0] B,     // 输入的乘数 B
    output [31:0] HI,   // 高 32 位结果
    output [31:0] LO    // 低 32 位结果
);

/* 内部信号 */
wire [63:0] result_mux;           // 统一存储乘法结果
wire [63:0] result_unsigned;     // 无符号乘法结果
wire signed [63:0] result_signed; // 有符号乘法结果
wire [63:0] A_unsigned_ext;      // 扩展的无符号 A
wire [63:0] B_unsigned_ext;      // 扩展的无符号 B
wire signed [63:0] A_signed_ext;  // 扩展的有符号 A
wire signed [63:0] B_signed_ext;  // 扩展的有符号 B

/* 扩展输入 */
assign A_unsigned_ext = { 32'd0, A };
assign B_unsigned_ext = { 32'd0, B };
assign A_signed_ext = {{32{A[31]}} , A};
assign B_signed_ext = {{32{B[31]}} , B};

/* 乘法运算 */
assign result_unsigned = A_unsigned_ext * B_unsigned_ext;
assign result_signed = A_signed_ext * B_signed_ext;
assign result_mux = sign_flag ? result_signed : result_unsigned;

/* 输出结果 */
assign HI = result_mux[63:32];
assign LO = result_mux[31:0];

endmodule

```


CP0, 协处理器, 用于处理中断或异常

```
module CP0(
    input cp0_clk,
    input cp0_rst,
    input cp0_ena,
    input MFC0,
    input MTC0,
    input ERET,
    input [31:0] PC,
    input [31:0] addr,
    input [4:0] cause,
    input [31:0] data_in,
    output[31:0] CP0_out,
    output[31:0] EPC_out
);

parameter SYSCALL = 5'b01000,    // SYSCALL
          BREAK   = 5'b01001,    // BREAK
          TEQ     = 5'b01101;    // TEQ

parameter STATUS = 4'd12,
          CAUSE   = 4'd13,
          EPC     = 4'd14;

reg [31:0] cp0_reg [0:31];    // 声明 CP0 处理器的内部寄存器堆
integer i;

assign CP0_out = (MFC0 && cp0_ena) ? cp0_reg[addr[4:0]] : 32'hz;
assign EPC_out = (ERET && cp0_ena) ? cp0_reg[EPC] : 32'hz;

always @(posedge cp0_clk or posedge cp0_rst) begin
    if (cp0_rst && cp0_ena) begin
        // 复位时将所有寄存器清零
        for ( i = 0; i < 32; i = i + 1) begin
            cp0_reg[i] <= 32'h0;
        end
    end else if (cp0_ena) begin
        // 处理 MTC0 指令
        if (MTC0) begin
            cp0_reg[addr[4:0]] <= data_in;
        end
    end
end
```

```

// 处理中断相关指令
else if (cause == SYSCALL || cause == BREAK || cause == TEQ) begin
    cp0_reg[STATUS] <= cp0_reg[STATUS] << 5;
    cp0_reg[CAUSE] <= {24'b0, cause, 2'b0};
    cp0_reg[EPC] <= PC;
end
// 处理 ERET 指令
else if (ERET) begin
    cp0_reg[STATUS] <= cp0_reg[STATUS] >> 5;
end
end
end

endmodule

```

CLZ, 用于计算输入数值前导 0 的个数

```

module CLZ(
    input [31:0] CLZ_in,    // 要计算前导 0 的数值
    output [31:0] CLZ_out   // 输出前导 0 的个数
);

reg [31:0] CLZ_out;

always @(*) begin

    if (CLZ_in == 32'b0) begin
        CLZ_out = 32;
    end else begin // 初始化计数器
        integer i;
        integer count;
        count = 0;
        // 从最高位开始检查，直到找到第一个非零位
        for (i = 31; i >= 0; i = i - 1) begin
            if (CLZ_in[i]) begin
                // 找到第一个非零位，退出循环
                CLZ_out = count;
                break;
            end
            // 否则，前导 0 的数量加 1
            count = count + 1;
        end
    end
end

```

```

        end
    end

endmodule

```

Divider, 分频器

```

module Divider(clk,rst_n,clk_out);
input clk;                //系统时钟
input rst_n;              //复位信号,低电平有效
output reg clk_out;        //输出适配 CPU 的时钟

parameter DIVISOR = 50000000;//50,000,000 分频
reg [31:0] count=32'd0;
always @(posedge clk)
begin
    if(!rst_n)//复位, count, clk_out 清零
    begin
        count <= 1'b0;
        clk_out <= 0;
    end
    else if(count == DIVISOR)
    begin
        count <= 32'd0;
        clk_out <= ~clk_out;
    end
    else
        count <= count+1'b1;
end
endmodule

```

seg7x16, 七段数码管驱动

```

module seg7x16(
    input clk,
    input reset,
    input cs,
    input [31:0] i_data,
    output [7:0] o_seg,
    output [7:0] o_sel
);

```

```

reg [11:0] cnt;
always @ (posedge clk, posedge reset)
  if (reset)
    cnt <= 0;
  else
    cnt <= cnt + 1'b1;

wire seg7_clk = cnt[11];

reg [2:0] seg7_addr;

always @ (posedge seg7_clk, posedge reset)
  if(reset)
    seg7_addr <= 0;
  else
    seg7_addr <= seg7_addr + 1'b1;

reg [7:0] o_sel_r;

always @ (*)
  case(seg7_addr)
    7 : o_sel_r = 8'b01111111;
    6 : o_sel_r = 8'b10111111;
    5 : o_sel_r = 8'b11011111;
    4 : o_sel_r = 8'b11101111;
    3 : o_sel_r = 8'b11110111;
    2 : o_sel_r = 8'b11111011;
    1 : o_sel_r = 8'b11111101;
    0 : o_sel_r = 8'b11111110;
  endcase

reg [31:0] i_data_store;
always @ (posedge clk, posedge reset)
  if(reset)
    i_data_store <= 0;
  else if(cs)
    i_data_store <= i_data;

reg [7:0] seg_data_r;
always @ (*)
  case(seg7_addr)
    0 : seg_data_r = i_data_store[3:0];
    1 : seg_data_r = i_data_store[7:4];

```

```

        2 : seg_data_r = i_data_store[11:8];
        3 : seg_data_r = i_data_store[15:12];
        4 : seg_data_r = i_data_store[19:16];
        5 : seg_data_r = i_data_store[23:20];
        6 : seg_data_r = i_data_store[27:24];
        7 : seg_data_r = i_data_store[31:28];
    endcase

    reg [7:0] o_seg_r;
    always @ (posedge clk, posedge reset)
        if(reset)
            o_seg_r <= 8'hff;
        else
            case(seg_data_r)
                4'h0 : o_seg_r <= 8'hC0;
                4'h1 : o_seg_r <= 8'hF9;
                4'h2 : o_seg_r <= 8'hA4;
                4'h3 : o_seg_r <= 8'hB0;
                4'h4 : o_seg_r <= 8'h99;
                4'h5 : o_seg_r <= 8'h92;
                4'h6 : o_seg_r <= 8'h82;
                4'h7 : o_seg_r <= 8'hF8;
                4'h8 : o_seg_r <= 8'h80;
                4'h9 : o_seg_r <= 8'h90;
                4'hA : o_seg_r <= 8'h88;
                4'hB : o_seg_r <= 8'h83;
                4'hC : o_seg_r <= 8'hC6;
                4'hD : o_seg_r <= 8'hA1;
                4'hE : o_seg_r <= 8'h86;
                4'hF : o_seg_r <= 8'h8E;
            endcase

    assign o_sel = o_sel_r;
    assign o_seg = o_seg_r;

endmodule

```

testbench 文件:

```

module cpu_tb;
    reg clk;           //时钟信号
    reg rst;           //复位信号

```

```

wire [31:0] inst;    //要执行的指令
wire [31:0] pc;      //下一条指令的地址
reg  [31:0] cnt;     //计数器，已经执行了几条指令
integer file_open;

initial
begin
    clk = 1'b0;
    rst = 1'b1;
    #50 rst = 1'b0;
    cnt = 0;
end

always #10 clk = ~clk;

always @ (posedge clk) begin
    cnt <= cnt + 1'b1;
    file_open = $fopen("D://test", "a+");
    $fdisplay(file_open, "OP: %d", cnt);
    $fdisplay(file_open, "Instr_addr = %h", sc_inst.inst);
    $fdisplay(file_open, "$zero = %h", sc_inst.sccpu.cpu_ref.array_reg[0]);
    $fdisplay(file_open, "$at   = %h", sc_inst.sccpu.cpu_ref.array_reg[1]);
    $fdisplay(file_open, "$v0   = %h", sc_inst.sccpu.cpu_ref.array_reg[2]);
    $fdisplay(file_open, "$v1   = %h", sc_inst.sccpu.cpu_ref.array_reg[3]);
    $fdisplay(file_open, "$a0   = %h", sc_inst.sccpu.cpu_ref.array_reg[4]);
    $fdisplay(file_open, "$a1   = %h", sc_inst.sccpu.cpu_ref.array_reg[5]);
    $fdisplay(file_open, "$a2   = %h", sc_inst.sccpu.cpu_ref.array_reg[6]);
    $fdisplay(file_open, "$a3   = %h", sc_inst.sccpu.cpu_ref.array_reg[7]);
    $fdisplay(file_open, "$t0   = %h", sc_inst.sccpu.cpu_ref.array_reg[8]);
    $fdisplay(file_open, "$t1   = %h", sc_inst.sccpu.cpu_ref.array_reg[9]);
    $fdisplay(file_open, "$t2   = %h", sc_inst.sccpu.cpu_ref.array_reg[10]);
    $fdisplay(file_open, "$t3   = %h", sc_inst.sccpu.cpu_ref.array_reg[11]);
    $fdisplay(file_open, "$t4   = %h", sc_inst.sccpu.cpu_ref.array_reg[12]);
    $fdisplay(file_open, "$t5   = %h", sc_inst.sccpu.cpu_ref.array_reg[13]);
    $fdisplay(file_open, "$t6   = %h", sc_inst.sccpu.cpu_ref.array_reg[14]);
    $fdisplay(file_open, "$t7   = %h", sc_inst.sccpu.cpu_ref.array_reg[15]);
    $fdisplay(file_open, "$s0   = %h", sc_inst.sccpu.cpu_ref.array_reg[16]);
    $fdisplay(file_open, "$s1   = %h", sc_inst.sccpu.cpu_ref.array_reg[17]);
    $fdisplay(file_open, "$s2   = %h", sc_inst.sccpu.cpu_ref.array_reg[18]);
    $fdisplay(file_open, "$s3   = %h", sc_inst.sccpu.cpu_ref.array_reg[19]);
    $fdisplay(file_open, "$s4   = %h", sc_inst.sccpu.cpu_ref.array_reg[20]);
    $fdisplay(file_open, "$s5   = %h", sc_inst.sccpu.cpu_ref.array_reg[21]);
    $fdisplay(file_open, "$s6   = %h", sc_inst.sccpu.cpu_ref.array_reg[22]);
    $fdisplay(file_open, "$s7   = %h", sc_inst.sccpu.cpu_ref.array_reg[23]);

```

```

    $fdisplay(file_open, "$t8 = %h", sc_inst.sccpu.cpu_ref.array_reg[24]);
    $fdisplay(file_open, "$t9 = %h", sc_inst.sccpu.cpu_ref.array_reg[25]);
    $fdisplay(file_open, "$k0 = %h", sc_inst.sccpu.cpu_ref.array_reg[26]);
    $fdisplay(file_open, "$k1 = %h", sc_inst.sccpu.cpu_ref.array_reg[27]);
    $fdisplay(file_open, "$gp = %h", sc_inst.sccpu.cpu_ref.array_reg[28]);
    $fdisplay(file_open, "$sp = %h", sc_inst.sccpu.cpu_ref.array_reg[29]);
    $fdisplay(file_open, "$fp = %h", sc_inst.sccpu.cpu_ref.array_reg[30]);
    $fdisplay(file_open, "$ra = %h", sc_inst.sccpu.cpu_ref.array_reg[31]);
    $fdisplay(file_open, "$pc = %h\n", sc_inst.pc);
    $fclose(file_open);
end

sccomp_dataflow sc_inst(
    .clk_in(clk),
    .reset(rst),
    .inst(inst),
    .pc(pc)
);

endmodule

```